CS 530 Lab Notebook

Lab 5

Name: Jarren Calizo
OdinID: calizo
Date: November 2, 2025

Table of Contents

Lab Overview 3
Lab 5.1g Storage, IAM 4
Step 1: GCP Cloud Storage Overview 4
Step 2: GCP Cloud Storage #1 (USGS Earthquake VM) 4
1. What role is attached to the Compute Engine default service account? 5
2. Would it be sufficient for the VM to perform its functions (creating buckets and reading/writing objects)? 5
3. What permissions are given by the default access scope to Cloud Storage? 6
4. Would they be sufficient for the VM to perform its functions (creating buckets and reading/writing objects)? 7
5. What settings are possible for setting the VM’s access to the Storage API? 7
Step 3: Configuring Permissions 8
Step 4: USGS Data and Setup 9
1. What time did the latest earthquake happen? 9
2. What was the magnitude (mag)? 10
3. Where was the place it happened? 10
Step 5: Python Plotting Code 10
1. Take a screenshot of the image that has been created for your lab notebook. 11
Step 6: GCP Cloud Storage #2 (IAM Roles) 11
Step 7: Creating a Dedicated Service Account 12
Step 8: Creating a VM with the Custom Service Account 12
Step 9: Service Account Roles (Compute) 12
1. What is the exact error message that is returned? 13
2. What role needs to be added to the service account's permissions for the VM to have access to list the project's Compute Engine resources? 14
3. Take a screenshot of the output for your notebook. 14
Step 10: Service Account Roles (Storage) 15
1. What is the exact error message that is returned? 15
2. What role needs to be added to the service account's permissions for the VM to have access to add an object to a storage bucket? 16
3. Take a screenshot of the output for your notebook. 17
Step 11: GCP Cloud Storage #3 (Python) 17
Step 12: Python Storage Code Validation 18
Step 13: View and Delete Object 19
1. Take a screenshot the shows the entire URL and the image that has been retrieved 19
Step 14: Cleanup 19
Lab 5.2a DynamoDB Guestbook 20
Step 1: DynamoDB Model Integration 20
Step 2: model_dynamodb.py Review 20
Step 3: Setting up the Flask Environment 21
Step 4: Configuring AWS Credentials 21
Step 5: Running and Testing the Guestbook Application 22
1. Take a screenshot of the output for your lab notebook 22
Step 6: Version 2: Ubuntu VM Docker 24

Step 7: Running the Dockerized Application 24
1. Take a screenshot of the output for your lab notebook. 25
Step 8: Pushing the Container Image 26
1. Take a screenshot of the container image on DockerHub. 27
Step 9: Version 3: AWS Cloud9 IDE 28
Step 10: Configure the Security Group 29
Step 11: Run the Application 29
1. Take a screenshot as before that shows your entry and the IP address in the URL bar. 30
Step 12: Version 4: AWS EC2 31
Step 13: Connect to the Instance 31
Step 14: Set Up the Instance 32
Step 15: Visit the Application 32
1. Take a screenshot as before that shows your entry and the IP address in the URL bar. 33
Step 16: View the Database 33
1. Take a screenshot as before that shows your entry and the IP address in the URL bar. 34
Step 17: Clean-up 34
Lab 5.2g Cloud Datastore Guestbook 35
Step 1: Cloud Datastore Overview 35
Step 2: model_datastore Setup 36
Step 3: Reviewing Datastore Model Logic 36
Step 4: Datastore Configuration 36
Step 5: Environment Setup and Dependencies 36
Step 6: Obtaining GCP Credentials 37
Step 7: Running the Application 37
1. Take a screenshot of the output for your lab notebook. 38
Step 8: Version 2: Ubuntu VM Docker 39
Step 9: Run the Application 40
1. Take a screenshot of the output for your lab notebook. 40
Step 10: Push the Container Image 41
1. Take a screenshot of the container image on DockerHub. 42
Step 11: Version 3: GCP Cloud Shell 42
Step 12: Run the Application 43
1. Take a screenshot as before that shows your entry and the URL bar. 44
Step 13: Version 4: GCP Compute Engine 44
Step 14: Set up the instance 45
Step 15: Visit the application 46
1. Take a screenshot as before that shows your entry and the IP address in the URL bar. 46
Step 16: View the database 46
1. Take a screenshot of all of the entries that have been added including their timestamps for your lab notebook. 46
Step 17: Clean up 47

Lab Overview

Lab 5 was a comprehensive exploration of cloud storage, identity and access management (IAM), and multi-cloud integration through both AWS and GCP ecosystems. The lab’s progression mirrored a real-world evolution of cloud services - from foundational permission models to scalable, containerized, and production-ready deployments. It emphasized the intertwined relationship between access control, compute resources, and managed storage platforms while showcasing the architectural symmetry that cloud-native systems strive for.

The journey began with Lab 5.1g (Storage, IAM), where I explored Google Cloud Storage’s layered permission system - balancing IAM roles with API access scopes. Using a Compute Engine instance, I connected to USGS earthquake data, visualized it through Python, and uploaded the results to a public storage bucket. The exercise illuminated the principle of least privilege and how even small misalignments between IAM roles and OAuth scopes could break functionality. Creating a custom service account (gcs-lab) with progressively refined roles like Storage Object Viewer and Storage Object Creator reinforced secure design by minimizing overexposure of privileges. Later steps bridged manual IAM work with automated Python SDK operations, closing the loop from infrastructure to application-level integration.

The focus then shifted to Lab 5.2a (AWS DynamoDB Guestbook), marking a conceptual pivot from storage management to serverless NoSQL backends. Here, the familiar Flask Guestbook application evolved through several environments: a local Flask instance, a Dockerized container, AWS Cloud9, and finally an EC2 deployment. Each stage introduced greater abstraction and autonomy, replacing hardcoded credentials with role-based access and IAM instance profiles. Seeing four “Hello” entries accumulate - each from a different environment - was a powerful reflection of persistence design in cloud systems. It was no longer just an app; it became a distributed service resilient to deployment context.

The lab culminated in Lab 5.2g (Cloud Datastore Guestbook), where I mirrored the AWS architecture in Google Cloud. The Guestbook backend transitioned seamlessly to Google Cloud Datastore through minimal configuration changes - highlighting the power of modular design. Successive deployments (local VM, Docker, Cloud Shell, and Compute Engine) demonstrated portability, scalability, and identity-bound access through service accounts. By the final step, every environment - from local dev to GCP Compute Engine - converged into one shared Datastore instance. Each “Hello” entry symbolized a milestone in secure and efficient cloud migration.

In essence, Lab 5 unified the principles of identity, automation, and abstraction. Across two major cloud providers, I witnessed how the same architectural philosophy - isolating privilege, containerizing state, and binding compute to identity - produces security and scalability in tandem. This lab wasn’t just a checklist of tasks; it was a reflection on the architecture of trust, showing that true mastery of the cloud lies not in the platforms themselves, but in understanding how identity governs every action within them.

Lab 5.1g Storage, IAM

Step 1: GCP Cloud Storage Overview

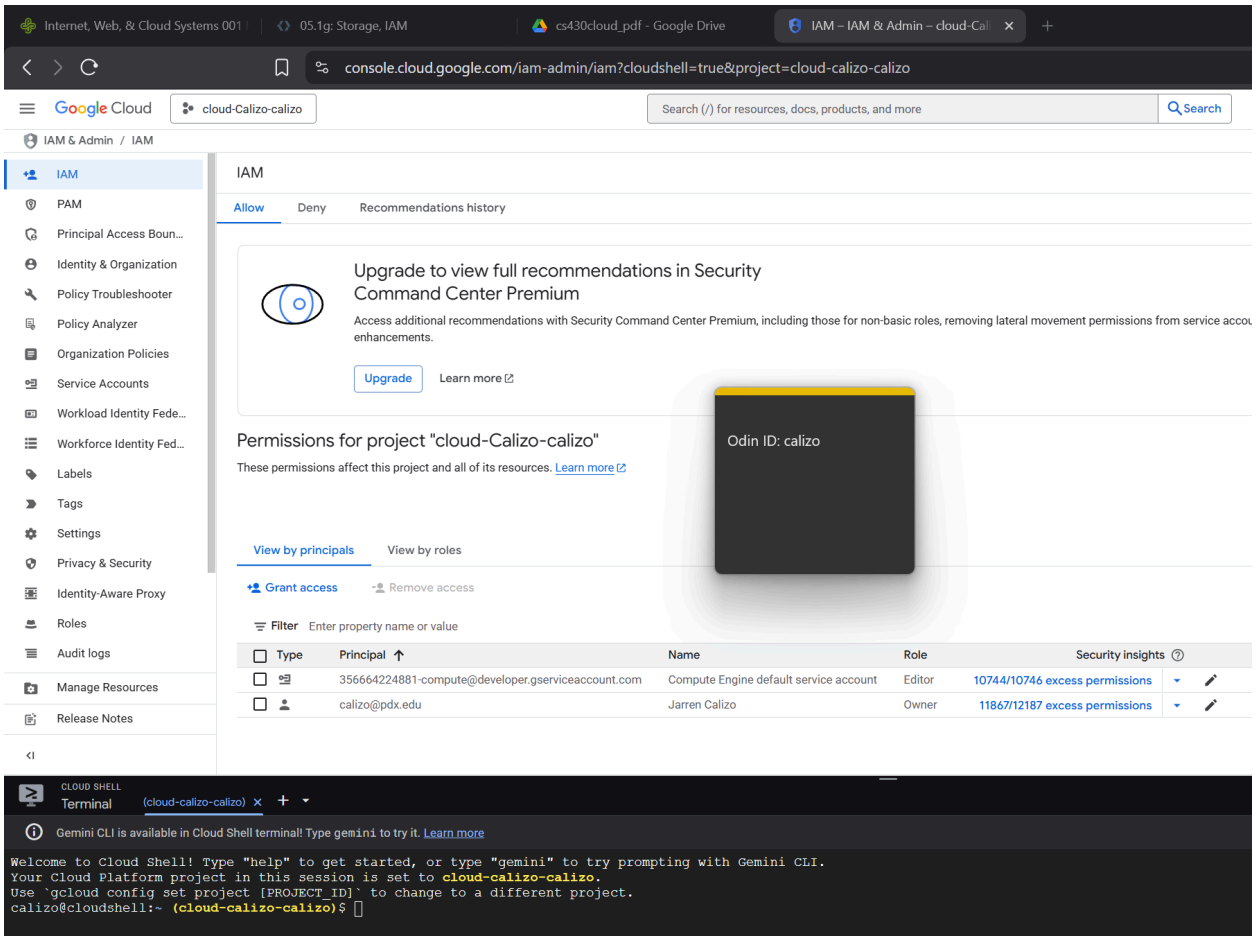
Google Cloud Storage (GCS) represents the heart of Google's object-based "storage-as-a-service" model. Rather than maintaining disks or provisioning servers manually, GCS abstracts storage into buckets and objects, accessible via uniform gs:// URIs. This allows developers to focus on application logic and data workflows rather than infrastructure overhead.

In this initial phase, I reviewed how GCS automatically handles replication and durability across regions, guaranteeing high availability. This sets the foundation for integrating with compute services such as Compute Engine, where virtual machines (VMs) can interact with storage buckets for data processing or public distribution of generated artifacts (like images, logs, or reports).

Step 2: GCP Cloud Storage #1 (USGS Earthquake VM)

The second step focused on configuring a VM capable of downloading and processing USGS earthquake data, generating a visual output through a Python script, and then distributing that output via Cloud Storage. This required verifying both IAM roles and access scopes - two layers of permissions that define how a VM interacts with Google Cloud APIs.

1. What role is attached to the Compute Engine default service account?



From the IAM & Admin → IAM console, the Compute Engine default service account (356664224881-compute@developer.gserviceaccount.com) is assigned the Editor role at the project level.

This basic role grants broad permissions across the project, including read and write privileges to most Google Cloud resources.

- Answer: The Compute Engine default service account has the Editor (roles/editor) role.

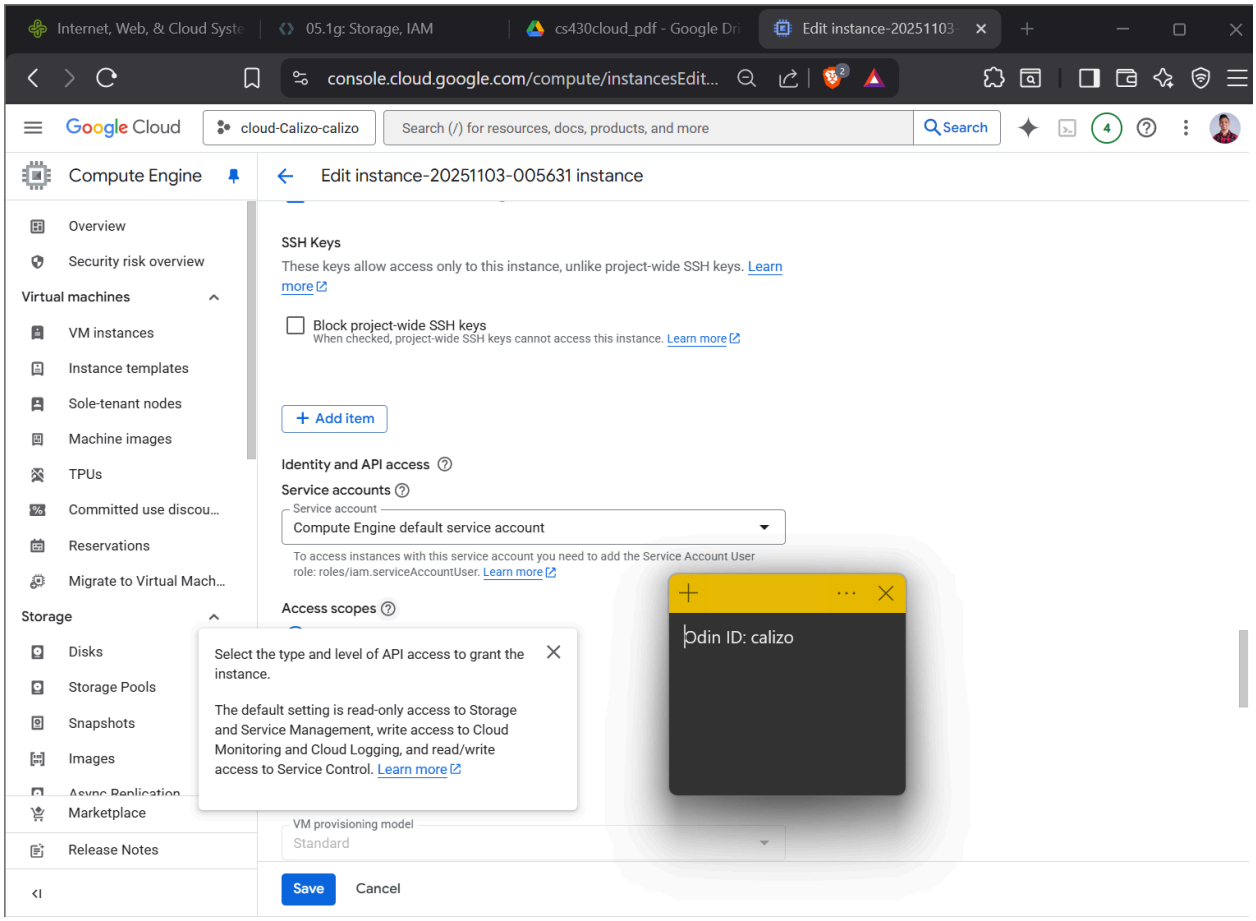
2. Would it be sufficient for the VM to perform its functions (creating buckets and reading/writing objects)?

Yes, the Editor role allows full bucket creation, listing, and object manipulation in Cloud Storage.

However, it is overly permissive - violating the principle of least privilege. A more secure approach would be to create a custom service account or assign a narrower role such as Storage Admin (roles/storage.admin), which grants only the permissions needed for bucket and object operations.

- Answer: Yes, sufficient but not ideal - least privilege recommends a smaller scope role.

3. What permissions are given by the default access scope to Cloud Storage?



When inspecting Access Scopes in the VM configuration, the tooltip reveals that the default access setting provides only read-only access to Storage and Service Management. This is enforced through the OAuth scope https://www.googleapis.com/auth/devstorage.read_only.

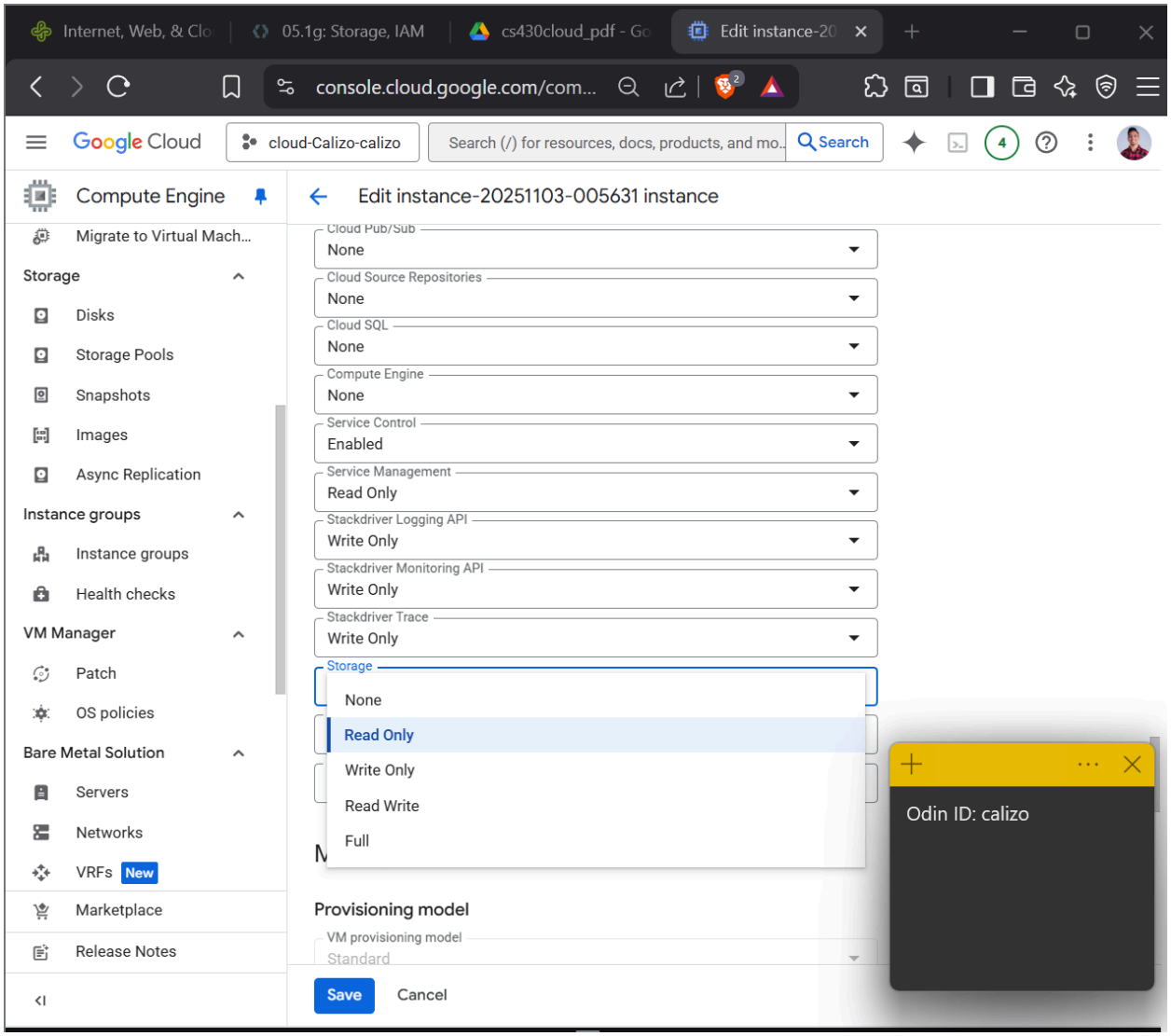
- Answer: The default access scope gives read-only access to Cloud Storage.

4. Would they be sufficient for the VM to perform its functions (creating buckets and reading/writing objects)?

No. Read-only access prevents both object uploads and bucket creation. To fulfill the lab's data-generation and upload tasks, the instance needs at least read/write or full control access scopes, paired with a service account that has the necessary IAM permissions.

- Answer: No, the default (read-only) scope is insufficient for bucket creation and uploads.

5. What settings are possible for setting the VM's access to the Storage API?



Scrolling down under "Set access for each API" shows a drop-down list for Storage, with five possible options:

1. None - no access to the API
2. Read Only - can only download objects or list buckets
3. Write Only - can upload but not read or list
4. Read Write - allows object reads/writes and bucket modifications
5. Full - complete control, equivalent to the cloud-platform scope

- Answer: Possible Storage API access-scope settings are None, Read Only, Write Only, Read Write, and Full.

Insight

This exercise clarified the two-layer permission model in GCP:

- IAM roles determine what actions a service account can perform across resources.
- Access scopes determine which APIs the instance can call at runtime.

In practice, both must align - a misconfigured scope can block even an IAM-approved action. Understanding this interplay is vital for secure cloud operations, especially when designing systems that automate storage interactions through Compute Engine.

Step 3: Configuring Permissions

The lab transitioned from conceptual IAM understanding to a practical configuration of VM access control. Rather than creating a separate service account, I instantiated a Compute Engine VM named usgs using the storage-full access scope, allowing full Cloud Storage API permissions.

The VM creation was performed through Cloud Shell with the following command:

```
gcloud compute instances create usgs \
  --machine-type=e2-medium --zone=us-west1-b \
  --image-project=ubuntu-os-cloud --image-family=ubuntu-2204-lts \
  --scopes=storage-full
```

This configuration preserved the Compute Engine default service account but granted it the Storage API privileges necessary to create buckets and read/write objects. After the VM appeared under the Compute Engine console, I accessed it via the SSH button to begin internal configuration.

The Cloud Shell terminal confirmed successful provisioning, signaling readiness for the subsequent earthquake data workflow.

Step 4: USGS Data and Setup

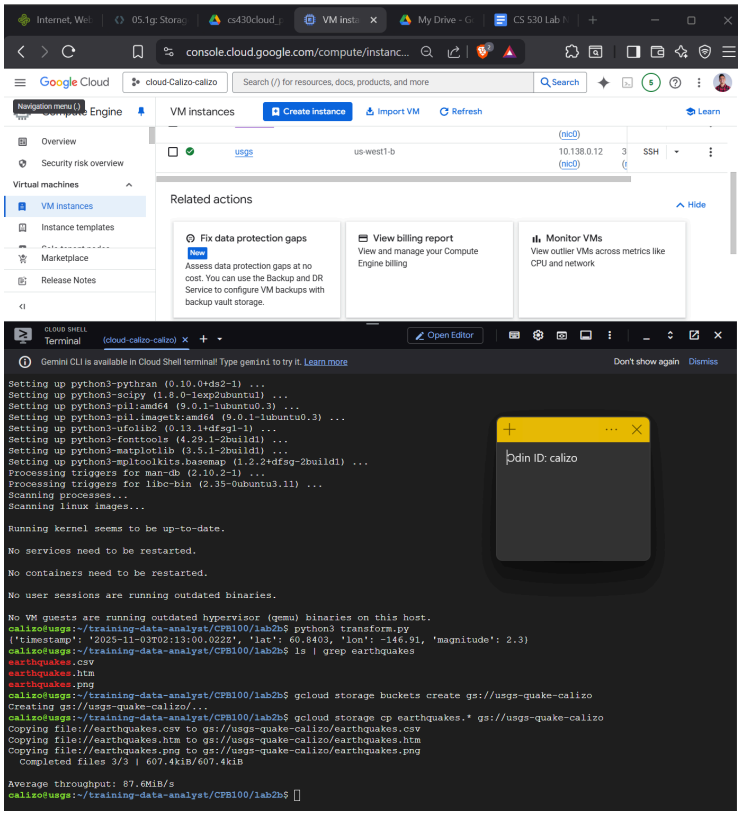
Once connected to the VM, I cloned the Google Cloud training-data-analyst repository, which contains the code resources for the lab. Navigating to the correct subdirectory prepared the workspace for data ingestion:

- git clone https://github.com/GoogleCloudPlatform/training-data-analyst
- cd training-data-analyst/CPB100/lab2b

The latest earthquake feed from the U.S. Geological Survey (USGS) was fetched directly as a CSV file and stored locally under earthquakes.csv:

- wget https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_week.csv -O earthquakes.csv
- head -2 earthquakes.csv

The first two rows of the file revealed real-time seismic activity data.



Using the provided output:

1. What time did the latest earthquake happen?

The CSV indicated the most recent quake occurred at 2025-11-03T10:03:00.022Z.

2. What was the magnitude (mag)?

The magnitude recorded was 2.3.

3. Where was the place it happened?

The event occurred near Alaska, aligning with the longitude and latitude observed in the file header output.

To prepare for the transformation script, Python dependencies were installed, including matplotlib, numpy, and requests, which enabled geospatial plotting and data handling. The installation logs confirmed the setup was complete and system binaries were updated successfully.

Step 5: Python Plotting Code

With all prerequisites in place, I executed the following command to generate a visual plot of the recent earthquakes worldwide:

- python3 transform.py

This script ingested the CSV file, parsed each earthquake's coordinates and magnitude, and created a global map visualization named earthquakes.png. Verifying with ls confirmed the presence of multiple output files (.csv, .png, .htm, and .json).

To distribute these results, I created a unique Cloud Storage bucket:

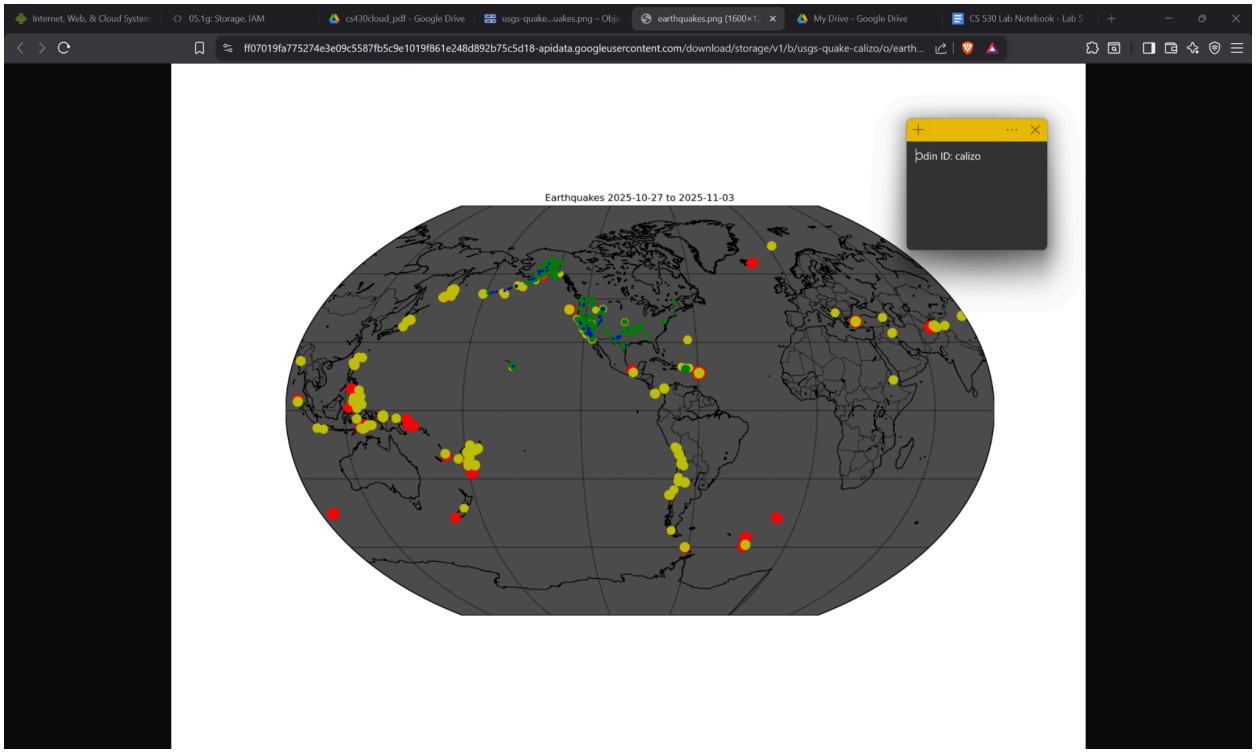
- gcloud storage buckets create gs://usgs-quake-calizo

and uploaded the output files:

- gcloud storage cp earthquakes.* gs://usgs-quake-calizo

The Cloud Shell output verified successful transfers for all files, including earthquakes.png. Opening the file in the Cloud Storage console revealed a vivid world map, with each circle color-coded by magnitude - green for minor tremors, yellow for moderate quakes, and red for higher magnitudes. The visual highlighted Earth's restless crust, stretching across the Pacific Ring of Fire and continental boundaries.

1. Take a screenshot of the image that has been created for your lab notebook.



Insight

This phase of the lab tied together IAM configuration, data ingestion, and visual data analytics in a cloud-native workflow. Starting from permissions management, I progressed through dataset retrieval, environment preparation, and final publication through a public Cloud Storage bucket. Each step represented a layer of the cloud stack - identity, compute, data, and visualization - working in orchestration to deliver meaningful insights from real-world seismic data.

Step 6: GCP Cloud Storage #2 (IAM Roles)

At this stage, the focus shifted from legacy access scopes to modern IAM role-based permissions. Rather than limiting Cloud API access through instance scopes, I adopted the best practice of setting the scope broadly (cloud-platform) while applying fine-grained role restrictions through IAM. This approach allows consistent and auditable permission management across all services, reinforcing the principle of least privilege within Google Cloud.

Step 7: Creating a Dedicated Service Account

To apply this principle, I created a dedicated service account named gcs-lab, designed specifically for object-level access within Cloud Storage. Using Cloud Shell, the account was initialized and assigned the roles/storage.objectViewer role:

```
gcloud iam service-accounts create gcs-lab
gcloud projects add-iam-policy-binding $cloud-Calizo-calizo \
  --member
serviceAccount:gcs-lab@$cloud-Calizo-calizo.iam.gserviceaccount.com \
  --role roles/storage.objectViewer
```

This configuration ensured that the new service account could view but not modify objects in storage buckets. Once created, the account appeared under the IAM Service Accounts list, confirming successful provisioning.

Step 8: Creating a VM with the Custom Service Account

Next, I deployed a new VM named gcs-lab-vm within the same region as the previous instance (us-west1-b), this time specifying the gcs-lab service account and assigning the cloud-platform scope. The following command was executed in Cloud Shell:

```
gcloud compute instances create gcs-lab-vm \
  --machine-type e2-medium --zone us-west1-b \
  --image-project ubuntu-os-cloud --image-family ubuntu-2004-lts \
  --scopes cloud-platform \
  --service-account gcs-lab@$cloud-Calizo-calizo.iam.gserviceaccount.com
```

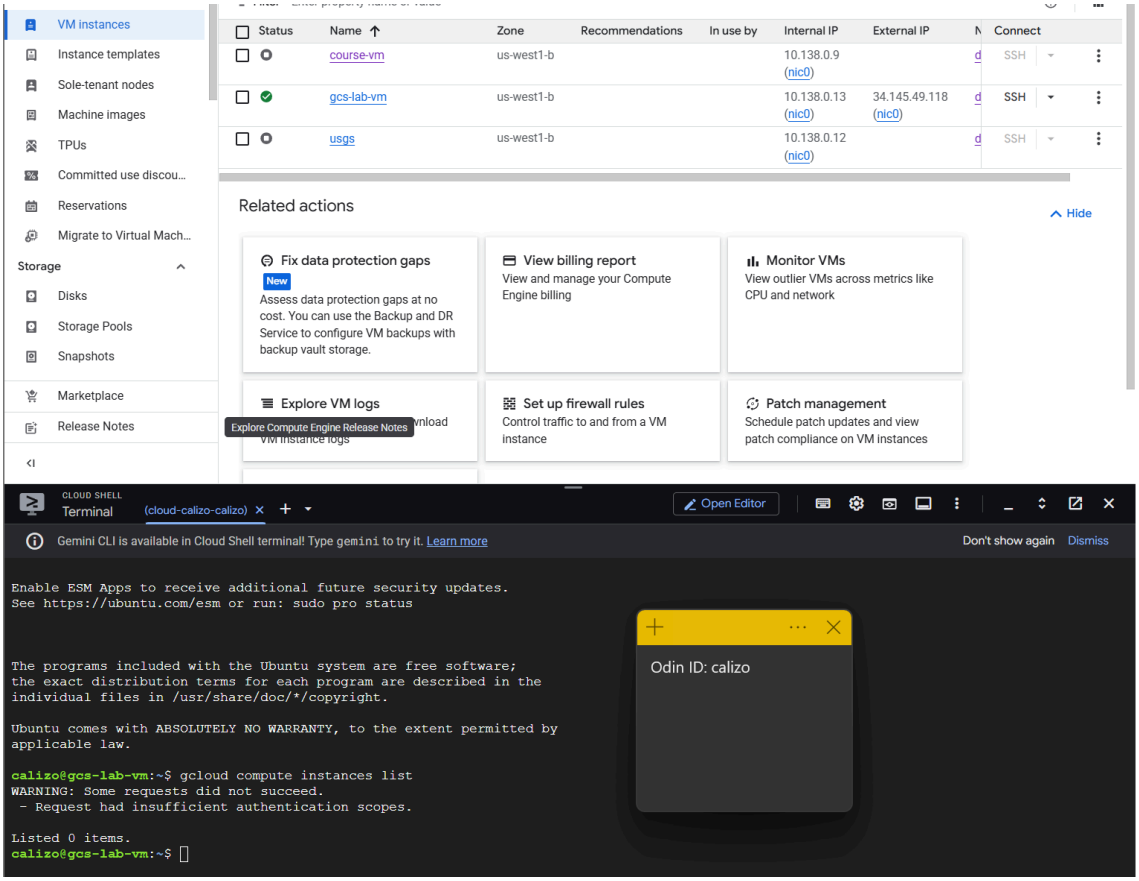
Upon completion, the Compute Engine dashboard reflected the new VM running alongside the prior usgs instance. Connecting to gcs-lab-vm via SSH initiated the verification phase.

Step 9: Service Account Roles (Compute)

Once inside the VM, I tested the account's permissions by attempting to list the available instances:

- gcloud compute instances list

1. What is the exact error message that is returned?



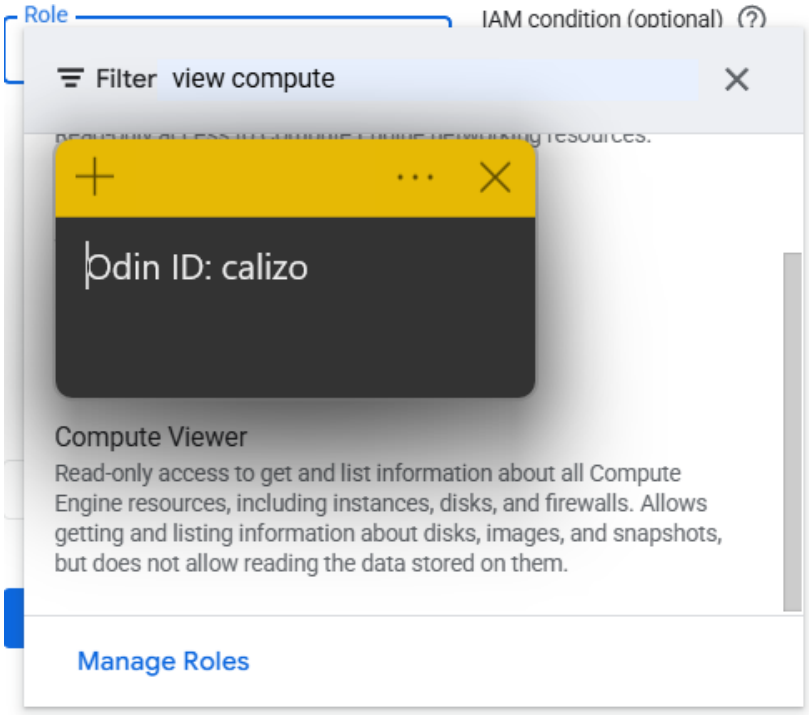
The command returned:

WARNING: Some requests did not succeed.
- Request had insufficient authentication scopes.

This confirmed that the Storage Object Viewer role alone did not provide sufficient privileges to query Compute Engine resources.

Returning to the IAM console, I edited the gcs-lab service account and added an additional role.

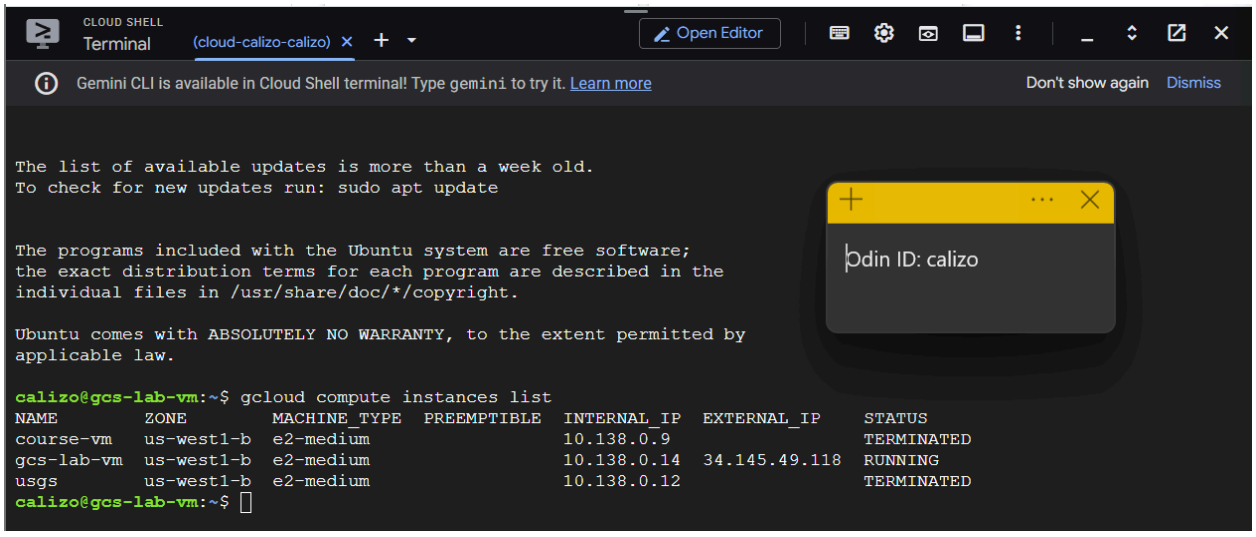
2. What role needs to be added to the service account's permissions for the VM to have access to list the project's Compute Engine resources?



The role added was Compute Viewer (roles/compute.viewer).

After saving the changes, I reconnected to the VM and reran the same command. This time, the output displayed all active and terminated instances within the project, including course-vm, usgs, and gcs-lab-vm.

3. Take a screenshot of the output for your notebook.



The terminal screenshot captured this listing, verifying the proper IAM binding.

Step 10: Service Account Roles (Storage)

The next portion tested the service account's Cloud Storage permissions. From the same gcs-lab-vm SSH session, I executed:

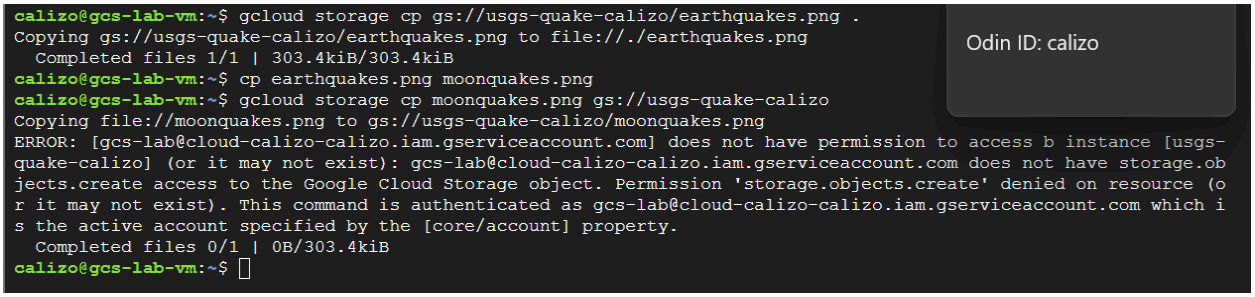
- gcloud storage cp gs://usgs-quake-calizo/earthquakes.png .

The command succeeded, confirming the Storage Object Viewer role allowed downloading objects.

I then attempted to rename and re-upload the file:

- cp earthquakes.png moonquakes.png
- gcloud storage cp moonquakes.png gs://usgs-quake-calizo/

1. What is the exact error message that is returned?



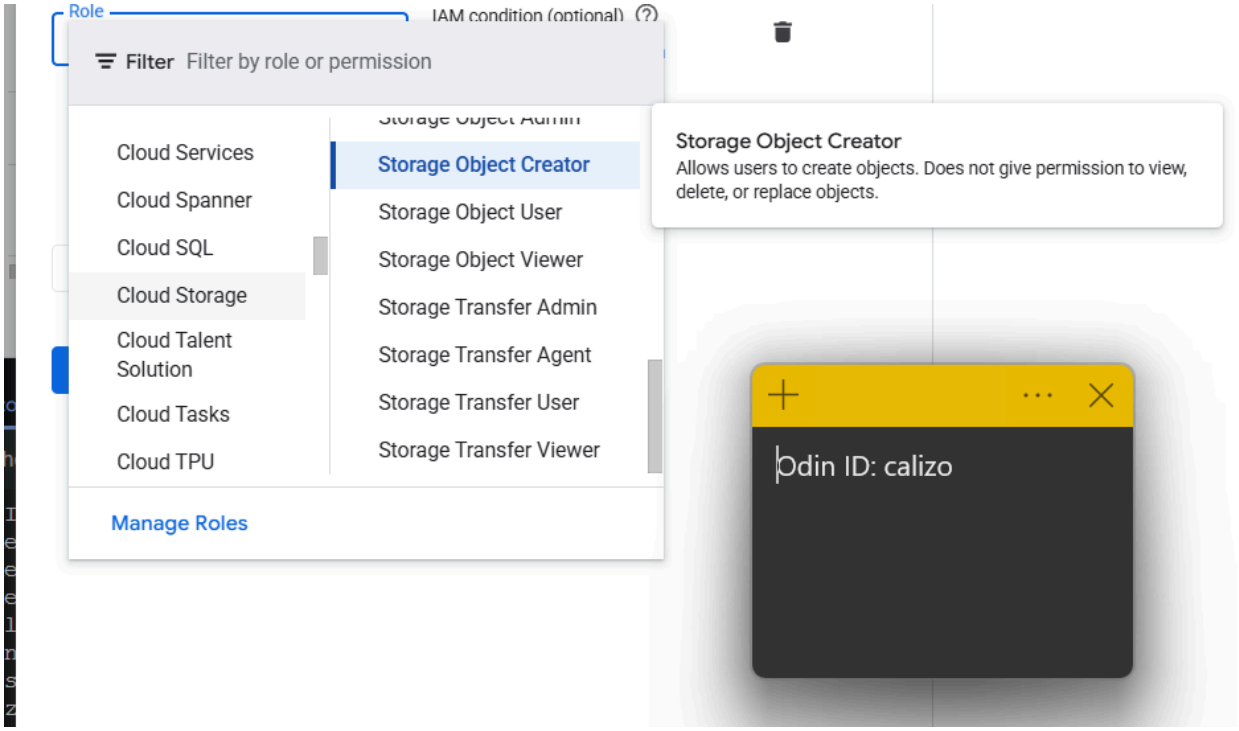
The attempt failed with the error:

> ERROR: [gcs-lab@cloud-calizo-calizo.iam.gserviceaccount.com] does not have permission
> to access b instance [usgs-quake-calizo] ... Permission 'storage.objects.create' denied
> on resource ...

This clarified that the account lacked write privileges for bucket uploads.

To resolve this, I returned to IAM → Service Accounts → gcs-lab → Edit and added an additional role.

2. What role needs to be added to the service account's permissions for the VM to have access to add an object to a storage bucket?



The role added was Storage Object Creator (roles/storage.objectCreator).

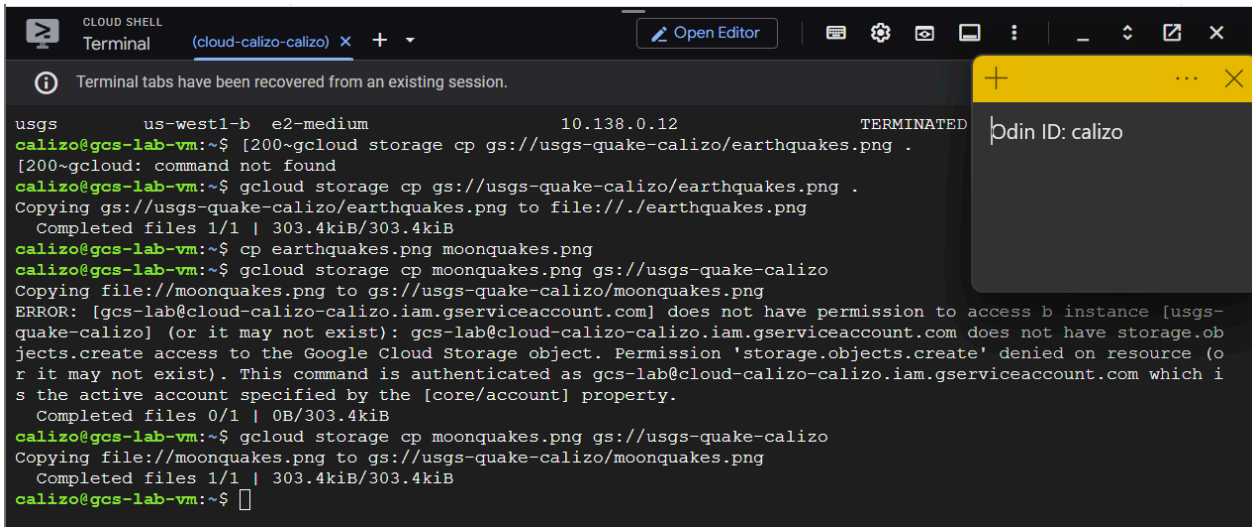
After saving and waiting for the changes to propagate, I repeated the upload command:

- gcloud storage cp moonquakes.png gs://usgs-quake-calizo/

The upload completed successfully, as shown in the terminal output:

- Copying file://moonquakes.png to gs://usgs-quake-calizo/moonquakes.png
- Completed files 1/1 | 303.4 KiB/303.4 KiB

3. Take a screenshot of the output for your notebook.



This screenshot displayed the successful file transfer back into the storage bucket, validating that the service account's new permissions allowed object creation without overexposing privileges to deletion or replacement.

Insight

This exercise reinforced the elegance of IAM-based security over access scopes, allowing granular control of who can view, who can modify, and who can create resources. By deliberately iterating through denied and approved permissions, I observed firsthand how the Storage Object Viewer and Storage Object Creator roles complement each other under least-privilege principles.

Together, these steps illustrated the maturity of Google Cloud's IAM model - precise, scalable, and easily auditable - aligning both with security best practices and operational efficiency.

Step 11: GCP Cloud Storage #3 (Python)

This final sequence transitioned from the CLI and IAM configuration work into programmatic interaction with Google Cloud Storage using the Python SDK. The objective was to simulate how an application could automate upload, sharing, and deletion operations through authenticated API calls.

In Cloud Shell, I downloaded one of the lab's preset images, selecting a motivational picture identified by its numbered index, and saved it locally as image.jpg. I then initialized a Python virtual environment, activated it, and installed the google-cloud-storage library to access GCS programmatically.

Once configured, I entered the Python interpreter and began with:

```
from google.cloud import storage
storage_client = storage.Client()
bucket = storage_client.get_bucket('usgs-quake-calizo')
blob = bucket.blob('gcs-lab-image.jpg')
```

I opened the downloaded file in binary mode, uploaded it to my bucket, and made it public through:

```
blob.upload_from_string(myImage.read(), content_type='image/jpeg')
blob.make_public()
blob.public_url
```

This returned a URL that exposed the uploaded object via HTTP.

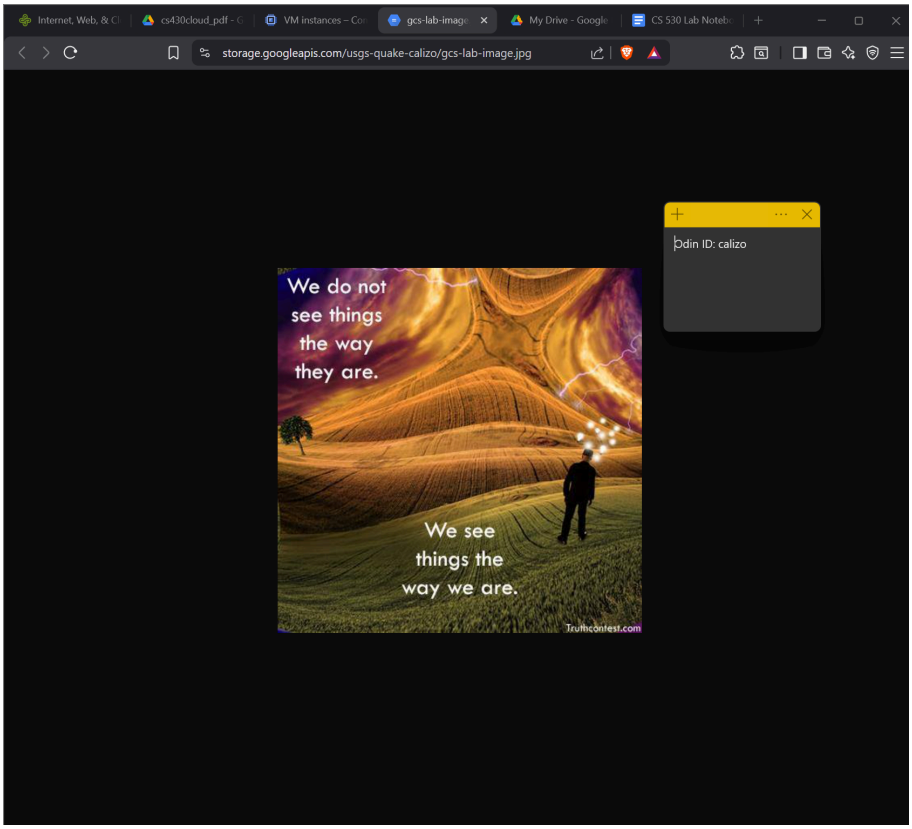
Step 12: Python Storage Code Validation

Within seconds, the bucket contained a new object titled gcs-lab-image.jpg. Copying the generated public URL into a browser opened the file directly from Google Cloud Storage (https://storage.googleapis.com/usgs-quake-calizo/gcs-lab-image.jpg). The screenshot verified this public object access.

The success confirmed that the Python SDK client operated correctly under the default credentials tied to Cloud Shell, and that the permissions were aligned for upload and public-read actions.

Step 13: View and Delete Object

1. Take a screenshot the shows the entire URL and the image that has been retrieved



The full URL and rendered image confirmed successful retrieval from the public endpoint.

Returning to the Python interpreter, I issued:

- blob.delete()

This deleted the uploaded image directly from the bucket through the API, verifying that Python client operations could perform both upload and removal of data objects.

Step 14: Cleanup

To prevent lingering cloud resources, I performed a complete teardown of all items created during Lab 5.1g. Using Cloud Shell, I sequentially executed the following commands:

gcloud storage rm gs://usgs-quake-calizo/*
gcloud compute instances delete usgs gcs-lab-vm --zone=us-west1-b

gcloud iam service-accounts delete
gcs-lab@$cloud-calizo-calizo.iam.gserviceaccount.com

The console confirmed the bucket deletion, instance terminations, and service account removal. All temporary resources were successfully deprovisioned.

Insight
Steps 11 through 14 embodied the practical essence of cloud-native integration - bridging IAM, storage, and application logic into a cohesive flow. Using Python’s GCS client library demonstrated how developers can script the same operations done manually earlier in the lab, but with far greater automation and flexibility. The final screenshot symbolized the lab’s overarching theme: perception and access are shaped by perspective - a fitting conclusion to an exercise grounded in control, visibility, and security within the cloud ecosystem.

Lab 5.2a DynamoDB Guestbook

Step 1: DynamoDB Model Integration

The first step involved pulling the updated course repository that introduced a new backend model implementation for AWS DynamoDB. After cloning and navigating into the 05_aws_dynamodb directory, the files revealed a clear architectural alignment with the prior Guestbook MVP structure, but this time the persistence layer pointed to DynamoDB through the updated gbmodel/__init__.py.

In this file, the model_backend was explicitly set to "dynamodb", which ensured that upon application startup, Flask would load the model class from model_dynamodb.py instead of the in-memory list version. This abstraction through a base Model class allowed seamless swapping of storage backends without altering the core app logic - a practical design principle in scalable cloud-native architectures.

Step 2: model_dynamodb.py Review

The new backend model defined within model_dynamodb.py was responsible for interfacing directly with the AWS DynamoDB service. Upon initialization, the code attempted to load an existing table named "guestbook", and if unavailable, it automatically created one.

The KeySchema used a composite key composed of:

- Partition key (HASH): email
- Sort key (RANGE): date

Both were of type String, with a minimal ProvisionedThroughput of 1 RCU/WCU, ideal for a low-traffic lab environment. The select() method used a scan() operation to retrieve all records, while insert() relied on put_item() to add new entries with name, email, date, and message fields. This configuration confirmed the Guestbook's transition to a true serverless NoSQL model.

Step 3: Setting up the Flask Environment

Within the Ubuntu VM, I activated an isolated Python virtual environment to ensure dependency consistency:

```
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
```

The requirements.txt file listed the familiar Flask microframework, boto3 for AWS SDK integration, and gunicorn as a WSGI HTTP server. With these dependencies installed, the local environment was primed for hybrid connectivity between the locally running web app and DynamoDB in the AWS cloud.

Step 4: Configuring AWS Credentials

To authenticate the local Guestbook to DynamoDB, I retrieved temporary AWS credentials from the AWS Academy Learner Lab interface. The panel revealed the Access Key ID, Secret Access Key, and Session Token, which were configured as environment variables in the VM terminal.

```
export AWS_ACCESS_KEY_ID=ASIA...
export AWS_SECRET_ACCESS_KEY=/T473...
export AWS_SESSION_TOKEN=IQoJ...
export AWS_DEFAULT_REGION=us-east-1
```

A quick verification using boto3.client("sts").get_caller_identity() confirmed valid identity recognition. The credentials successfully authorized outbound connections to DynamoDB within the us-east-1 region (N. Virginia), as shown in the screenshot of the AWS CLI credential setup.

Step 5: Running and Testing the Guestbook Application

Once authentication was in place, I launched the Flask application:

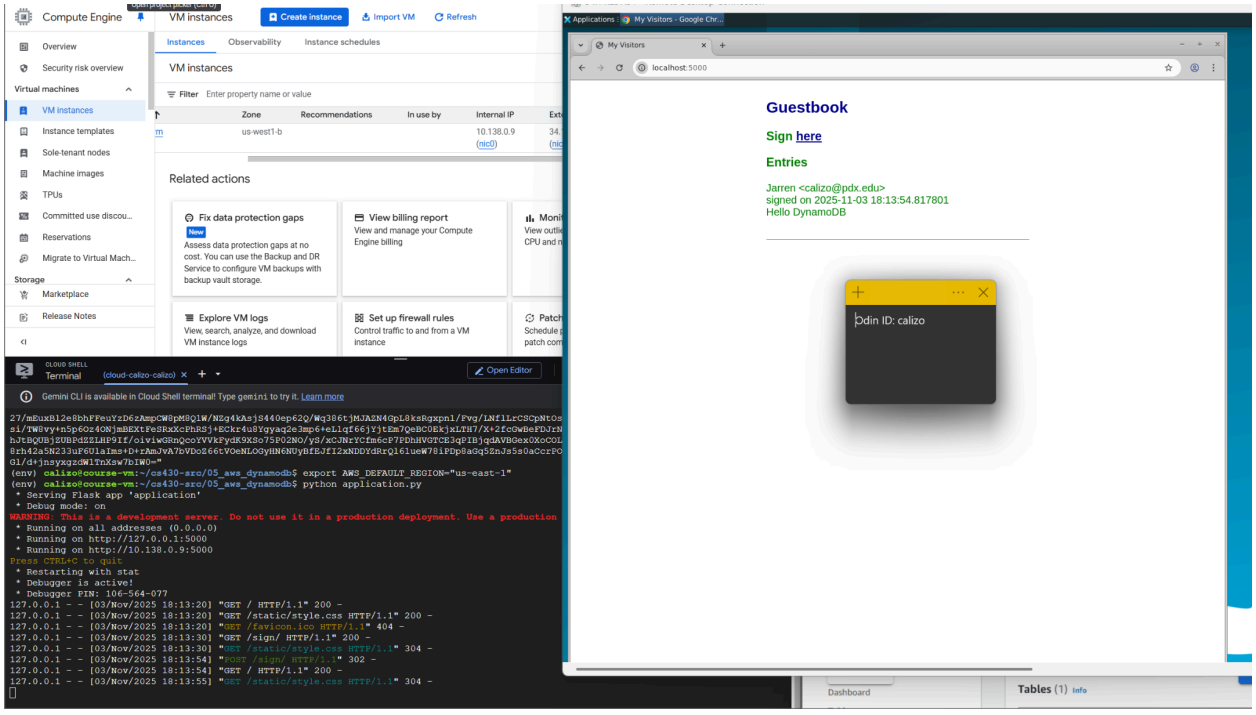
- python application.py

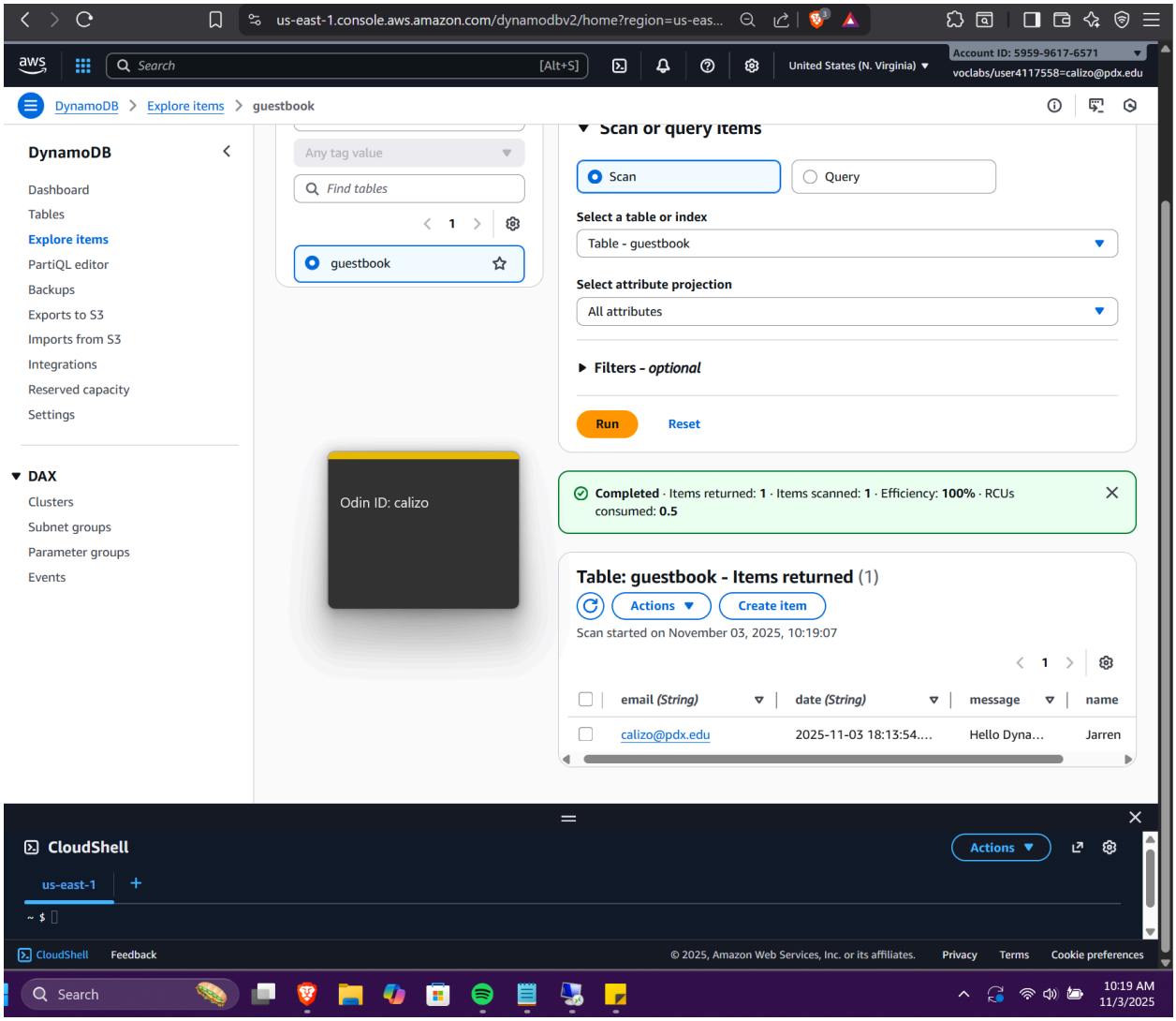
The Guestbook web app initialized on port 5000, displaying a familiar interface. I submitted my entry with:

- Name: Jarren
- Email: calizo@pdx.edu
- Message: "Hello DynamoDB"

The entry appeared immediately under the Entries section, confirming a successful round trip from frontend → Flask → DynamoDB backend.

1. Take a screenshot of the output for your lab notebook





The browser screenshot with the entry "Hello DynamoDB" and corresponding AWS Console view of the guestbook table satisfies this requirement

<u>Insight</u>

This portion of the lab elegantly demonstrated hybrid cloud integration - a local application seamlessly persisting data to a managed NoSQL service in AWS. By abstracting database operations behind an interchangeable backend model, the Guestbook evolved from a static prototype to a distributed, cloud-connected web system. The simplicity of the boto3 SDK and the elasticity of DynamoDB together highlighted one of cloud computing’s most empowering ideas: scaling functionality without scaling complexity.

Step 6: Version 2: Ubuntu VM Docker

After confirming the Python version of the Guestbook worked with DynamoDB, the next step was to containerize the application. This ensured portability and scalability across different environments.
Within the 05_aws_dynamodb directory, I examined the Dockerfile and updated the maintainer line to my PSU email. The container was built on the lightweight python:alpine base image and used Gunicorn to run Flask on port 80 or whichever port was passed via environment variable.

To prevent unnecessary files from bloating the image, a .dockerignore was also created to exclude the virtual environment and cached Python bytecode:

```
env
__pycache__
```

Building the container produced an image tagged aws_gb, ready to encapsulate the Guestbook's environment for deployment.

Step 7: Running the Dockerized Application

Using the AWS credentials from the Vocareum "AWS Details" panel, I passed the environment variables securely to the container (rather than embedding them in the image) and mapped the container's port 80 to the Ubuntu VM's port 8000.

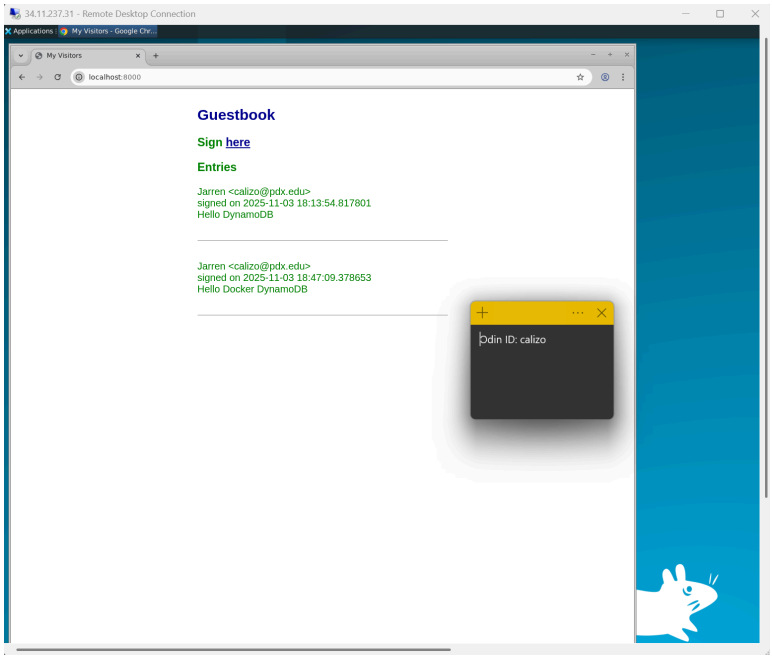
Docker run command executed:

```
docker run --env AWS_ACCESS_KEY_ID=... \
        --env AWS_SECRET_ACCESS_KEY=... \
        --env AWS_SESSION_TOKEN=... \
        --env AWS_DEFAULT_REGION=us-east-1 \
        --env PORT=80 \
        -p 8000:80 aws_gb
```

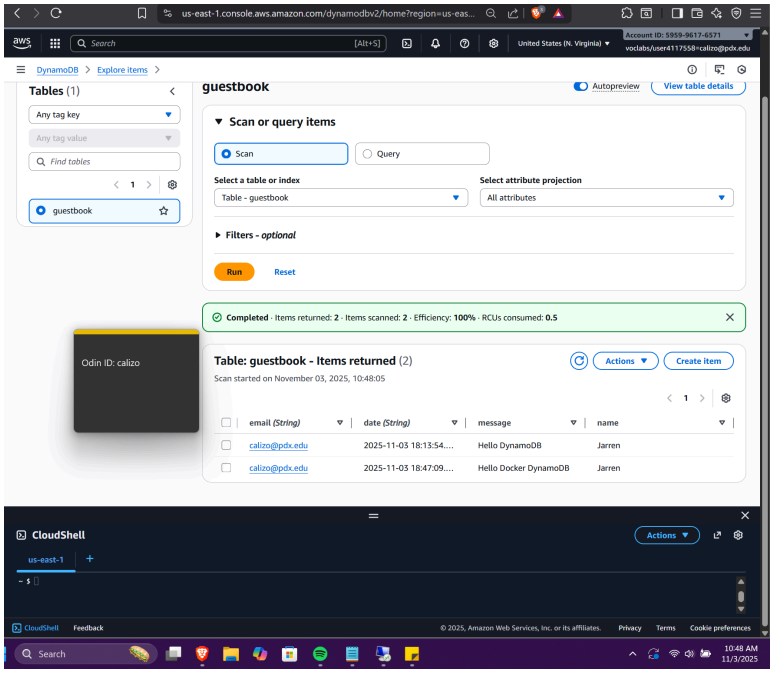
Gunicorn launched successfully, binding to port 80 and spawning worker threads for the Flask application. Opening http://localhost:8000 on the Ubuntu VM showed the Guestbook interface, where I signed again with:

- Name: Jarren
- Email: calizo@pdx.edu
- Message: "Hello Docker DynamoDB"

1. Take a screenshot of the output for your lab notebook.



The browser displayed both my first entry (“Hello DynamoDB”) from the earlier Python version and the new entry (“Hello Docker DynamoDB”), confirming persistent backend storage in DynamoDB.



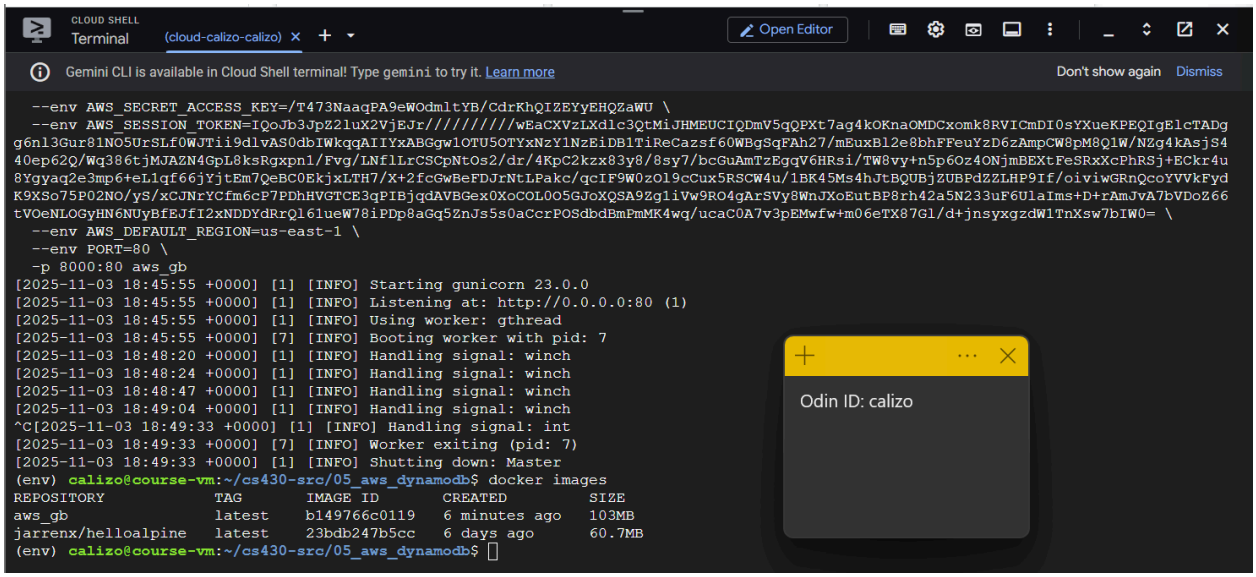
The AWS Console verified this with two total items in the guestbook table - each corresponding to one signed entry with unique timestamps.

After validation, I stopped the container using Ctrl+C. The consistent behavior across both versions confirmed that state was fully decoupled from the containerized runtime and persisted within AWS's serverless NoSQL backend.

Step 8: Pushing the Container Image

With the container working locally, the final step was to publish it to Docker Hub for portability and reuse.

1. Inspect image size:
   docker images

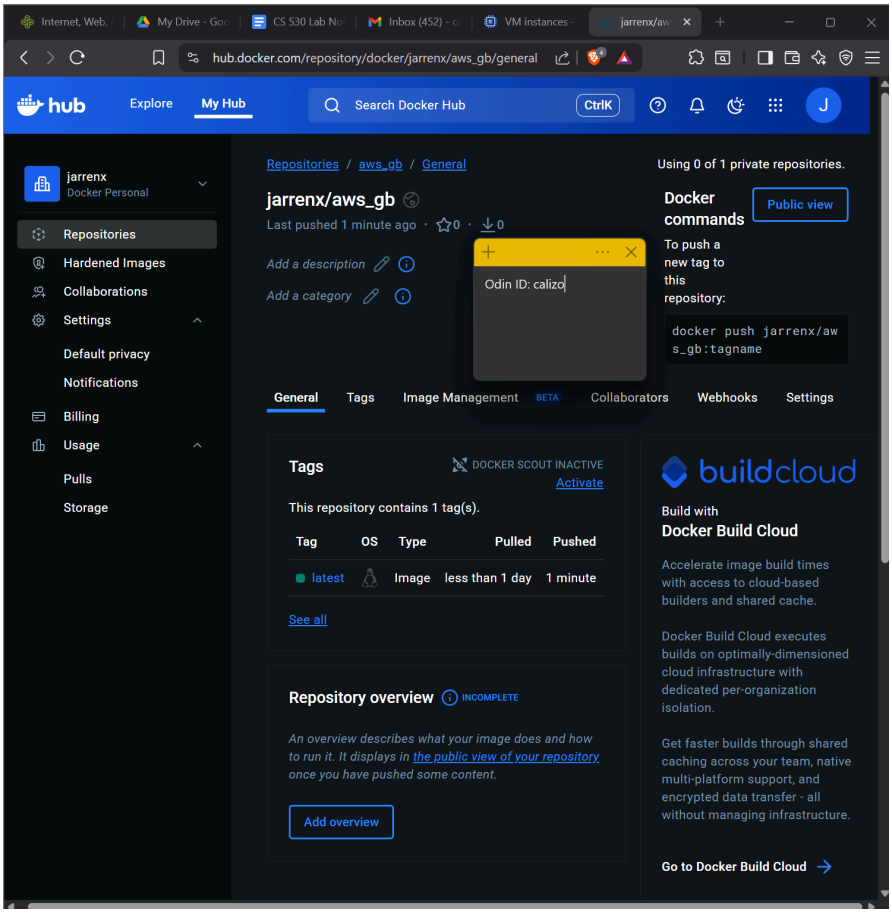


The aws_gb image was approximately 103MB, a compact footprint for a fully configured Python web app.

2. Tag and push to Docker Hub:
   docker login
   docker tag aws_gb jarrenx/aws_gb
   docker push jarrenx/aws_gb

Once uploaded, the repository became visible on Docker Hub under my account at: https://hub.docker.com/r/jarrenx/aws_gb

1. Take a screenshot of the container image on DockerHub.



Docker Hub confirmed the push was successful, showing the repository jarrenx/aws_gb with the latest tag, indicating it was publicly available for download and execution elsewhere.

Insight

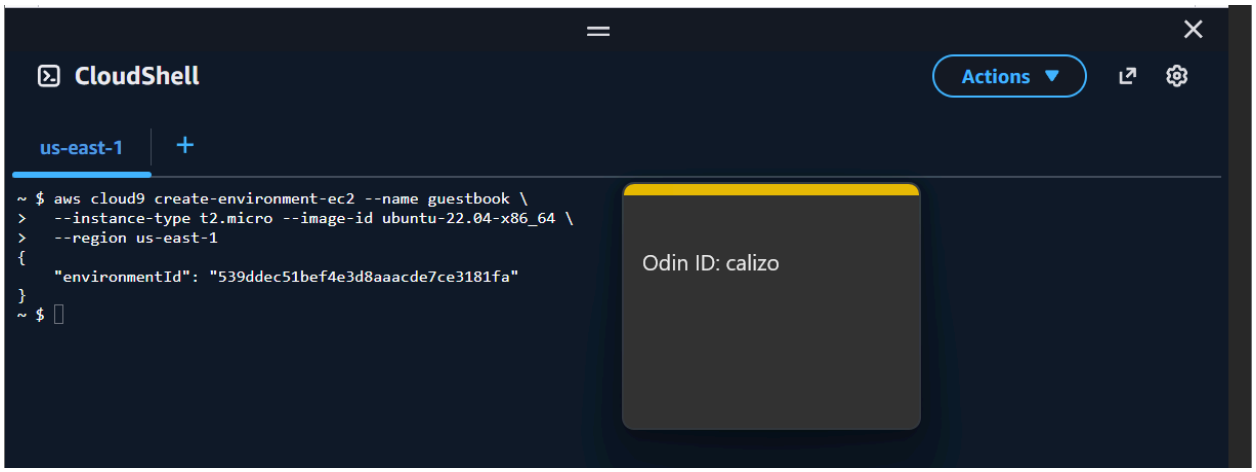
These steps embodied the bridge between development and deployment - transforming a locally running Python app into a portable, cloud-integrated container. The persistence of DynamoDB entries across both native and containerized runs illustrated the beauty of decoupled architectures: compute could move freely, while state lived securely in the cloud.

By the end of Step 8, the Guestbook was no longer just a Flask demo - it was a hybrid cloud application, ready for distribution through Docker Hub and scalable to any environment that supports container orchestration.

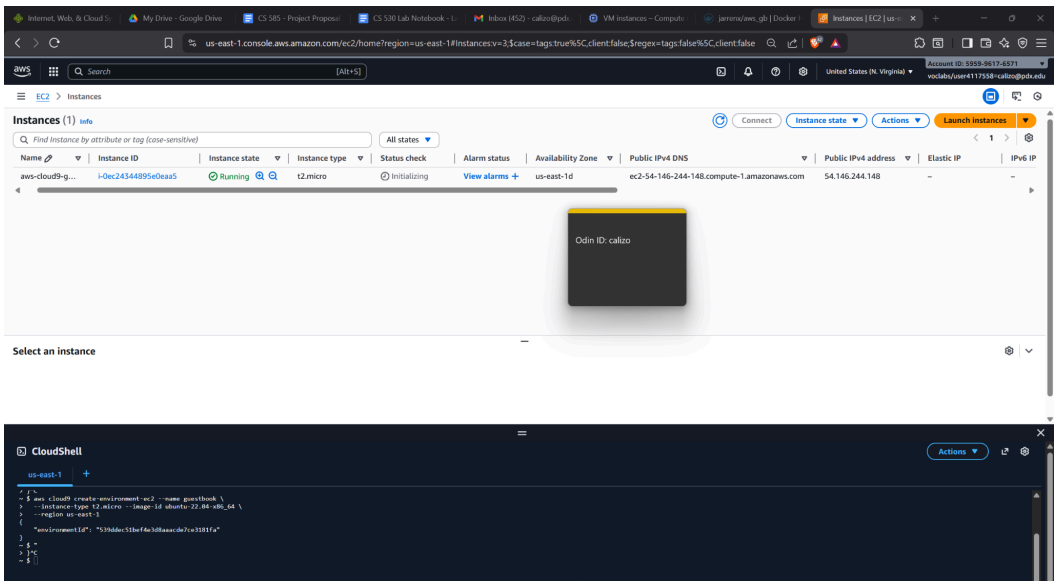
539ddec51bef4e3d8aaacde7ce3181fa

Step 9: Version 3: AWS Cloud9 IDE

For this stage, the Guestbook application evolved from local and Docker deployments to a fully cloud-native environment using AWS Cloud9. Cloud9 automatically provisions an EC2 instance with a service-linked IAM role, granting broad permissions for integrated AWS access.



Using Cloud Shell, I launched the new environment with the command shown in the first screenshot. The JSON response confirmed creation via the returned environment ID. Checking the EC2 console verified the environment's instance (aws-cloud9-guestbook) running on a t2.micro in the us-east-1 region. The instance listed its public IPv4 address 54.146.244.148, which would serve as the public endpoint for the Flask application.



Step 10: Configure the Security Group

By default, Flask listens on port 5000, so inbound access needed to be explicitly allowed for that port.

In the EC2 instance's security group settings, I added a new inbound rule:

- Type: Custom TCP
- Port: 5000
- Source: 0.0.0.0/0

This rule allowed the Guestbook to be reachable from any browser. The configuration effectively mimicked creating a firewall exception for the application while keeping the rest of the instance protected.

Step 11: Run the Application

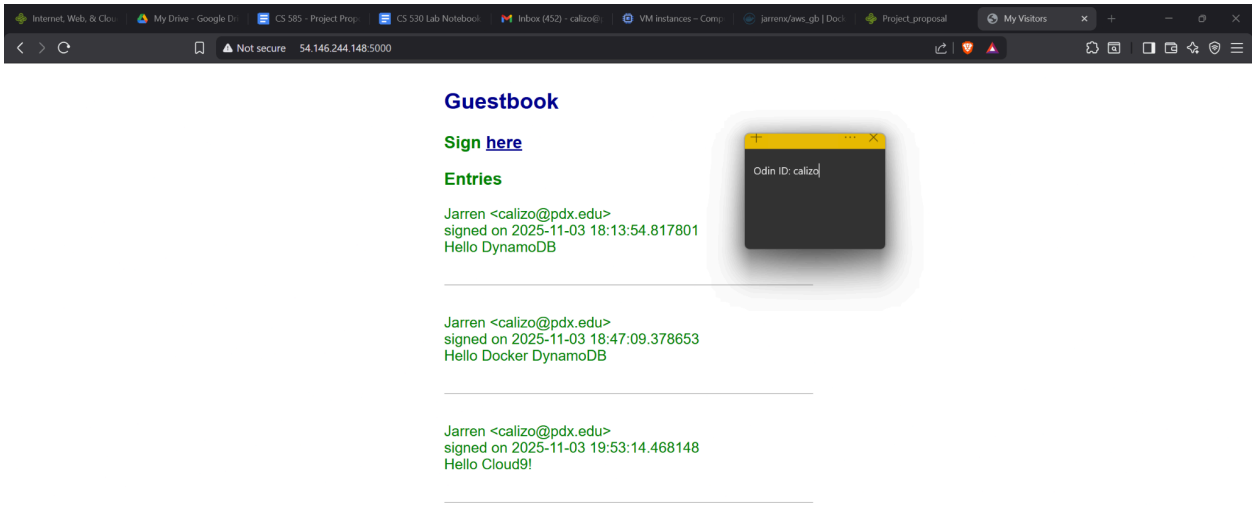
Inside the Cloud9 terminal, I cloned the class repository and switched into the DynamoDB lab folder:

```
git clone https://github.com/wu4f/cs430-src
cd cs430-src/05_aws_dynamodb
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
python application.py
```

Once Flask started serving on port 5000, I visited the instance's public IP in a local browser:

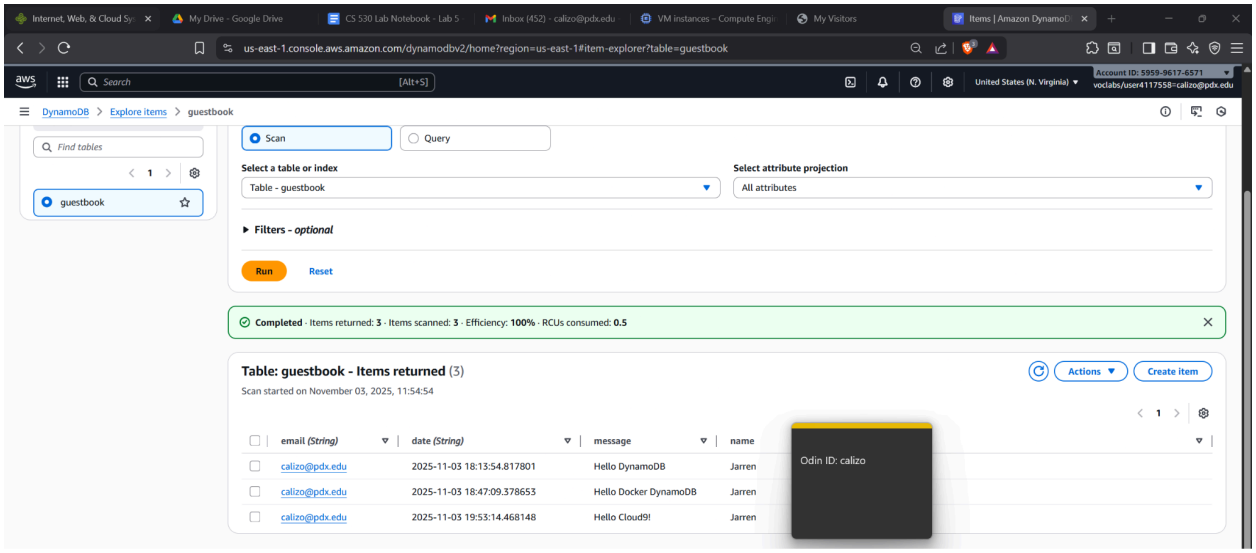
- http://54.146.244.148:5000

1. Take a screenshot as before that shows your entry and the IP address in the URL bar.



The Guestbook loaded seamlessly within Cloud9's environment, displaying all prior entries from previous versions:

- "Hello DynamoDB" (from the local run)
- "Hello Docker DynamoDB" (from the Docker version)
- And now a third entry, "Hello Cloud9!", successfully written from within the AWS Cloud9 IDE.



Finally, I verified the addition on the AWS Console. Opening DynamoDB → Tables → guestbook → Explore items showed all three entries persisted under the same table, with the latest timestamped record containing the Cloud9 message. This demonstrated that each environment - Ubuntu VM, Docker container, and Cloud9 IDE - shared the same backend datastore, validating consistent and secure cross-environment communication.

Insight
This transition to Cloud9 illustrated the progression from local development to an entirely cloud-based workflow. By running directly on AWS infrastructure with managed IAM permissions, deployment friction was eliminated. The persistent DynamoDB backend remained the single source of truth across all application variants, highlighting the modularity of the Guestbook's design.

Each entry across environments told a chronological story of the application's growth: from local roots to containerization, and finally, to the cloud itself.

Step 12: Version 4: AWS EC2

This final phase of the DynamoDB Guestbook lab transitions from a developer-centric cloud IDE to a production-oriented EC2 deployment. The goal was to host the same containerized Guestbook image on an independent EC2 instance that uses a role-based authentication model with minimal privileges.

I launched a new EC2 instance named guestbook running Ubuntu 22.04 (64-bit x86). During creation, I generated a key pair (awslab.pem) for secure SSH access, and configured the security group to allow SSH (port 22) and HTTP (port 80) traffic. In the advanced options, I assigned the LabInstanceProfile IAM role, ensuring that DynamoDB access would be handled automatically via instance metadata instead of static credentials.

The EC2 dashboard confirmed the instance running with the public IPv4 address 107.20.98.145, as shown in the first screenshot.

Step 13: Connect to the Instance

After downloading awslab.pem, I adjusted its permissions and connected to the instance via SSH from my local terminal:

```
cd ~/Downloads
chmod 400 awslab.pem
ssh -i awslab.pem ubuntu@107.20.98.145
```

The successful connection dropped me into the EC2 shell environment (ubuntu@ip-172-31-20-86), verifying key-based authentication and instance reachability.

Step 14: Set Up the Instance

Inside the VM, I installed Docker to prepare for containerized deployment:

```
sudo apt update -y
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

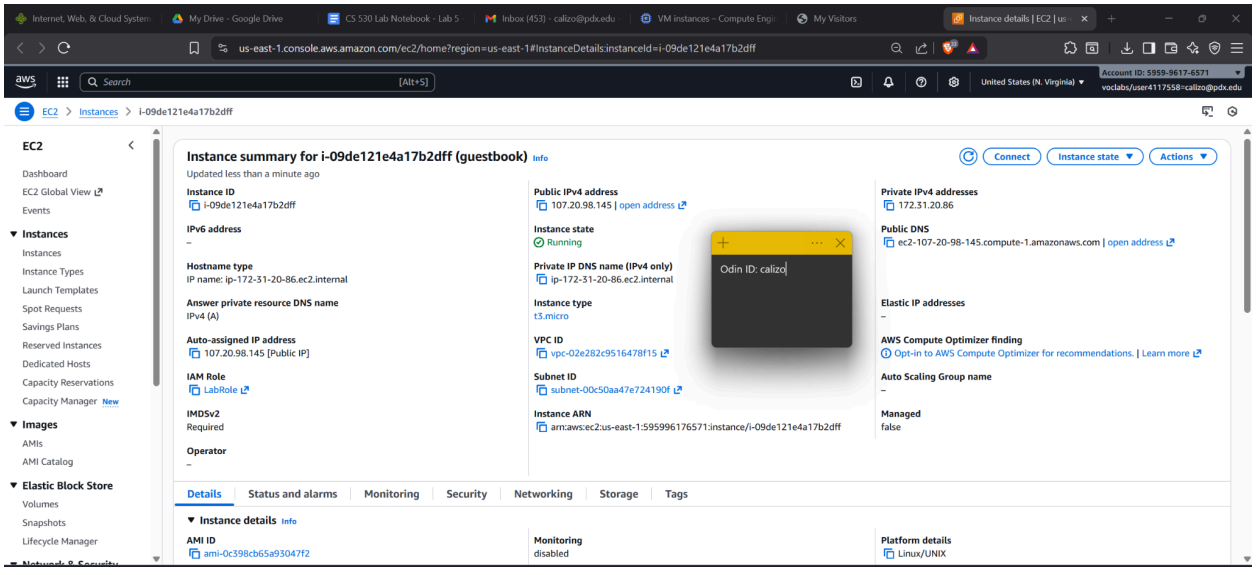
I then pulled and ran the Guestbook container image directly from DockerHub under my own namespace:

- sudo docker run --env AWS_DEFAULT_REGION=us-east-1 -p 80:80 jarrenx/aws_gb

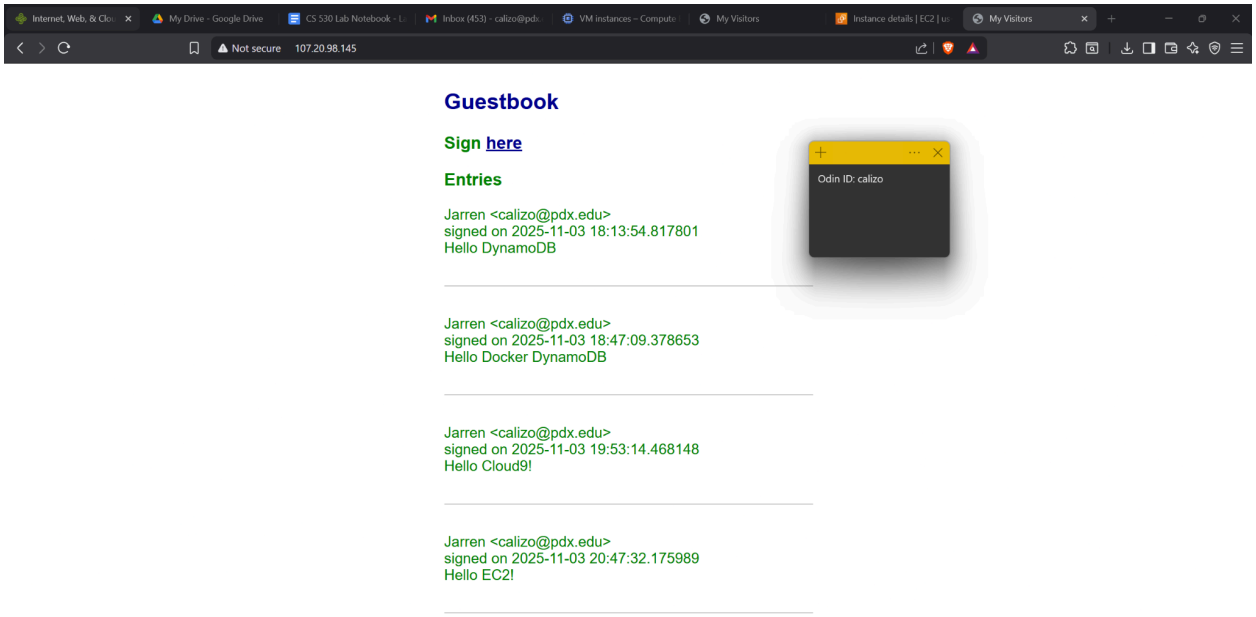
Because the instance already had the LabInstanceProfile attached, no AWS credentials were required to interact with DynamoDB. Docker automatically retrieved the latest image from DockerHub, and Flask (via Gunicorn) started listening on port 80. The terminal logs in the screenshot confirm container initialization and worker threads booting successfully.

Step 15: Visit the Application

Once the container was running, I visited the instance’s public IP address directly at: http://107.20.98.145



1. Take a screenshot as before that shows your entry and the IP address in the URL bar.



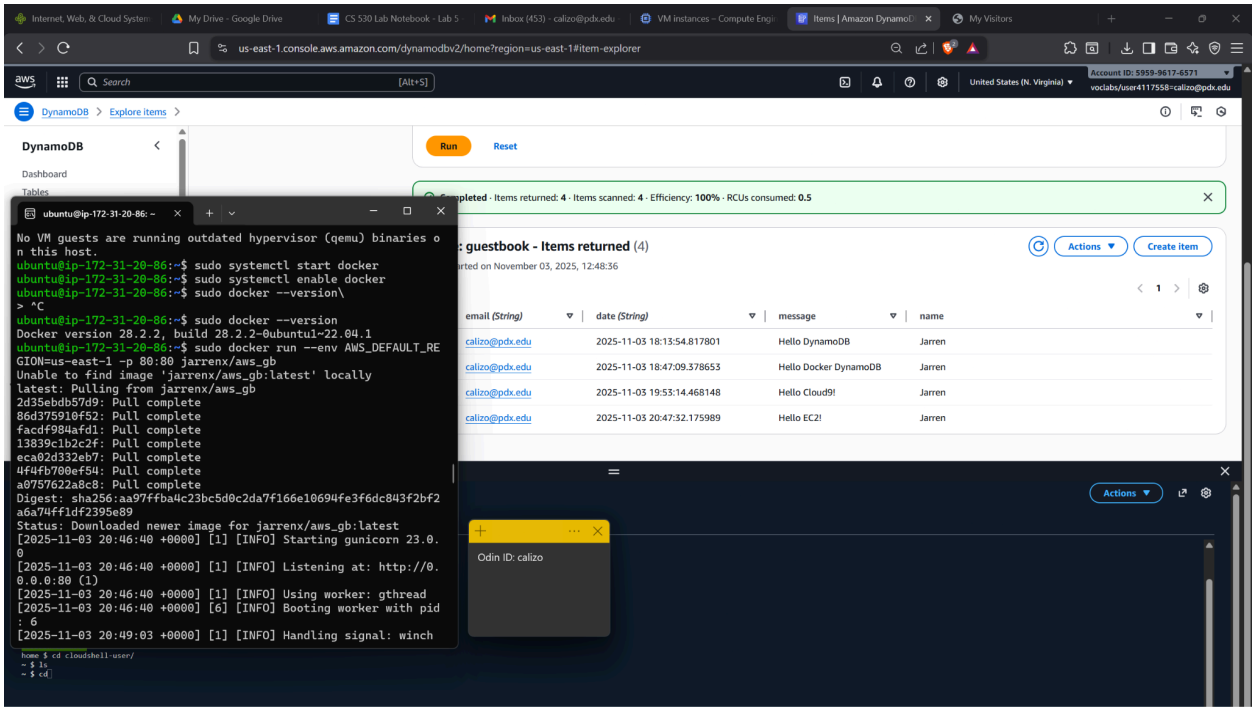
The Guestbook loaded in the browser, showing all prior entries from earlier environments. I added a new entry using my PSU email with the message "Hello EC2!" to mark the deployment's success.

Step 16: View the Database

Returning to the AWS DynamoDB Console → guestbook → Explore items, I verified that a fourth entry had been added. The table now contained all cumulative records across deployments, each with unique timestamps:

1. Hello DynamoDB
2. Hello Docker DynamoDB
3. Hello Cloud9!
4. Hello EC2!

1. Take a screenshot as before that shows your entry and the IP address in the URL bar.

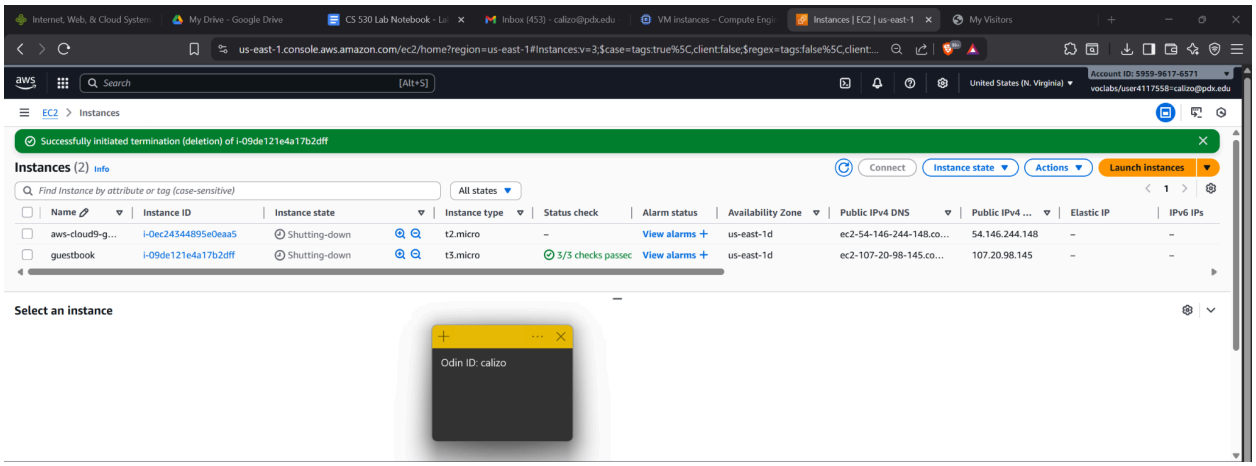


This DynamoDB table screenshot shows all four entries persisted correctly. This demonstrated that each environment - from local Flask to Docker to Cloud9 to EC2 - shared the same backend and remained consistent across all runs.

Step 17: Clean-up

To conclude, I cleaned up all cloud resources:

- Deleted the Cloud9 environment, which automatically terminated its linked EC2 instance.
- Terminated the guestbook EC2 instance manually from the EC2 console.



This screenshot shows both instances (aws-cloud9-guestbook and guestbook) entering a shutting-down state, confirming proper teardown and responsible cost management.

Final Insight

This final deployment felt like the culmination of a full-stack cloud engineering journey - starting locally, maturing into containerization, and finally achieving an independent EC2-hosted service bound by IAM-controlled access. Each successive environment served as a milestone of abstraction: from direct credentials to ephemeral tokens, and finally to a self-sufficient instance assuming its own role.

Seeing all four “Hello” entries united in DynamoDB embodied that progression - a single table chronicling the app’s evolution from development to production, proof that design modularity and cloud-native principles converged exactly as intended.

Lab 5.2g Cloud Datastore Guestbook

Step 1: Cloud Datastore Overview

This phase began by extending our Guestbook architecture into the realm of Google Cloud’s managed NoSQL backend. After cloning the repository and navigating into cs430-src/05_gcp_datastore, I verified that gbmodel/__init__.py correctly set model_backend = 'datastore'. This abstraction mirrored earlier lab architectures, proving how seamlessly one could swap storage implementations with minimal application code changes. The elegance of that modular design reflected a true model-view separation - one of the earliest hallmarks of scalable web engineering.

Step 2: model_datastore Setup

Inside gbmodel/model_datastore.py, I replaced YOUR_PROJECT_ID with my Google Cloud project identifier. Retrieving it through the Cloud Shell command echo $GOOGLE_CLOUD_PROJECT, I confirmed that the Datastore client now knew which project to bind to. This edit ensured all data transactions routed correctly to my GCP-managed backend instead of a local emulator. A subtle but vital step - without it, Flask would return 500 errors when inserting records into Datastore.

Step 3: Reviewing Datastore Model Logic

The next step involved confirming the three critical functions - select(), insert(), and from_datastore(). The select() method queried entities of kind Review, while from_datastore() reformatted raw Datastore entities into the list-of-lists structure expected by the Guestbook view. The insert() method generated a key, created an entity with name, email, message, and appended the timestamp via datetime.today(). Together, these functions established a full read-write flow to the Datastore backend without altering front-end logic - just as an abstraction layer should.

Step 4: Datastore Configuration

In the Google Cloud Console, I navigated to Datastore and initialized the service in Datastore Mode, ensuring the region was set to us-west. This was crucial since Firestore's native mode would diverge in schema behavior and indexing policies. With this setup complete, the backend was ready to receive structured entries under the kind Review, marking the transition from local storage to a globally replicated managed datastore.

Step 5: Environment Setup and Dependencies

Returning to my Ubuntu VM, I initialized a virtual environment and installed dependencies:

```
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
```

The requirements.txt file confirmed the app’s core components: Flask as the web framework, google-cloud-datastore for database connectivity, and Gunicorn for potential production deployment. The dependency installation completed without issue, signaling readiness for authentication and runtime tests.

Step 6: Obtaining GCP Credentials

To authenticate securely with Datastore, I created a service account named guestbook and assigned it the Datastore User role. After generating a JSON key, I transferred it to my VM, renamed it to guestbook-key.json, and exported its path as an environment variable:

- export GOOGLE_APPLICATION_CREDENTIALS=guestbook-key.json

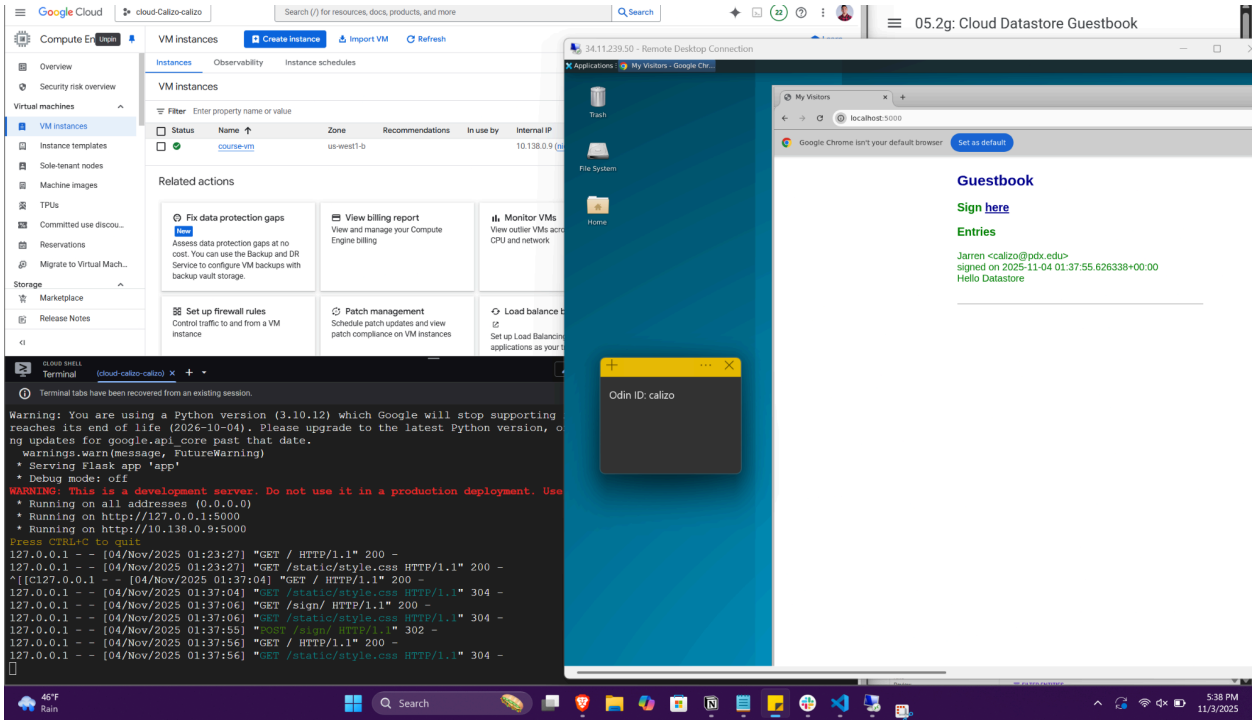
This ensured the Flask app could assume the proper identity when connecting to Google Cloud resources. By relying on IAM-managed credentials rather than hardcoded keys, the lab mirrored real-world security practices for distributed applications.

Step 7: Running the Application

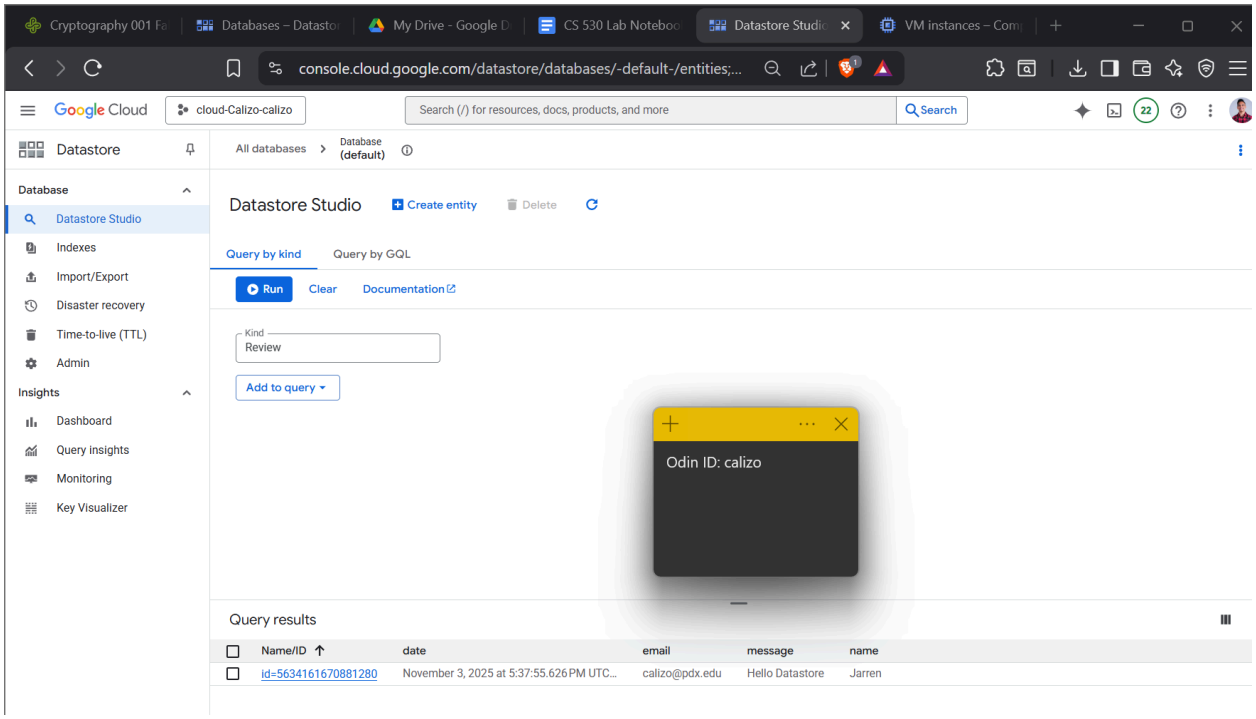
With all configuration steps complete, I launched the app:

- python app.py

1. Take a screenshot of the output for your lab notebook.



Flask initialized successfully on port 5000, as shown in the terminal output. Navigating to http://localhost:5000 opened the Guestbook, where I signed using my name and PSU email with the message "Hello Datastore".



Finally, returning to the Google Cloud Console under Datastore → Entities, I verified that the entry appeared within the Review kind - complete with name, email, date, and message fields. This completed the loop: a fully operational hybrid deployment bridging a local Python app and a managed cloud datastore backend.

Insight
This lab highlighted the quiet sophistication of abstraction in cloud design - how one could lift an application from SQLite to Google Cloud Datastore with only a few configuration changes. The simple green text of "Hello Datastore" on the Guestbook page symbolized more than connectivity; it was the visible endpoint of a carefully orchestrated backend migration. It marked the moment where local and cloud converged, not as separate systems, but as one seamless, federated application architecture.

Step 8: Version 2: Ubuntu VM Docker

After confirming that the Guestbook ran successfully on my local Python environment, I moved to containerization - a step that marked the transition from personal development to portable deployment. I opened the provided Dockerfile, examined the structure, and updated the maintainer field to my PSU email to reflect authorship. The base image google/cloud-sdk was intentionally large but valuable for its inclusion of precompiled dependencies like grpcio, saving time during the build process.

To prevent unnecessary local artifacts from bloating the image, I created a .dockerignore file containing env and __pycache__. This ensured that the container build remained clean and reproducible.

Building the image with:

```
sudo docker build -t gcp_gb .
```

produced a container ready to encapsulate the Flask app, Gunicorn server, and Datastore integration all within a consistent runtime environment. The transition from raw Python execution to containerization demonstrated the scalability mindset behind modern deployment - ensuring the same image could run identically on any host without dependency drift.

Step 9: Run the Application

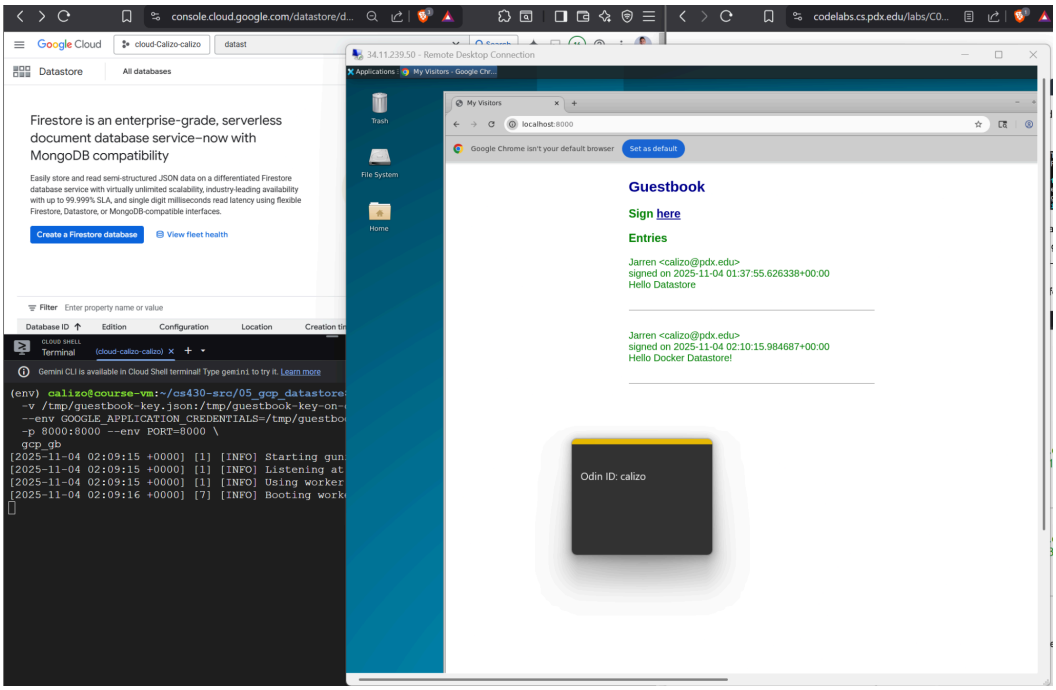
Once the image was built, I launched the container using the following configuration:

```
sudo docker run --rm \
  -v /tmp/guestbook-key.json:/tmp/guestbook-key-on-container.json:ro \
  --env GOOGLE_APPLICATION_CREDENTIALS=/tmp/guestbook-key-on-container.json \
  -p 8000:8000 --env PORT=8000 \
  gcp_gb
```

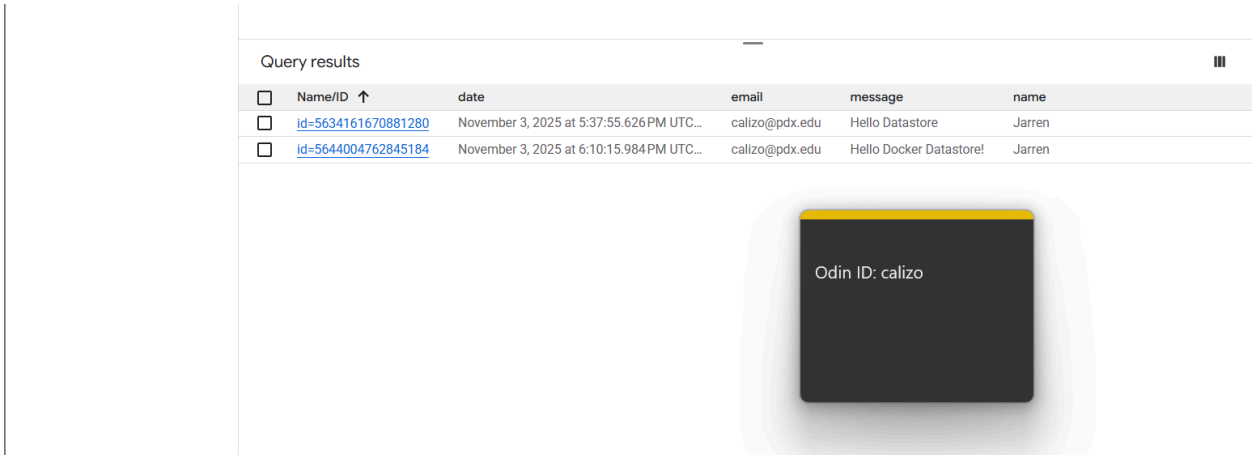
This command securely mounted my local service account key into the container as a read-only file and passed it as an environment variable so the application could authenticate with Google Cloud Datastore. The flags --rm and -p 8000:8000 ensured that the container auto-cleaned upon exit and bound port 8000 to my local VM for web access.

In the terminal logs, Gunicorn initialized successfully and confirmed the server was listening on http://0.0.0.0:8000. Visiting http://localhost:8000
 in the VM browser displayed the familiar Guestbook interface - this time containerized and powered through Gunicorn.

1. Take a screenshot of the output for your lab notebook.



I signed the Guestbook with the message "Hello Docker Datastore!" using my PSU email, and the new entry appeared directly beneath my previous "Hello Datastore" submission from Step 7.



Step 10: Push the Container Image

After verifying functionality, I checked the local Docker image size using:

- docker images

The tag gcp_gb appeared as expected. I then logged into Docker Hub:

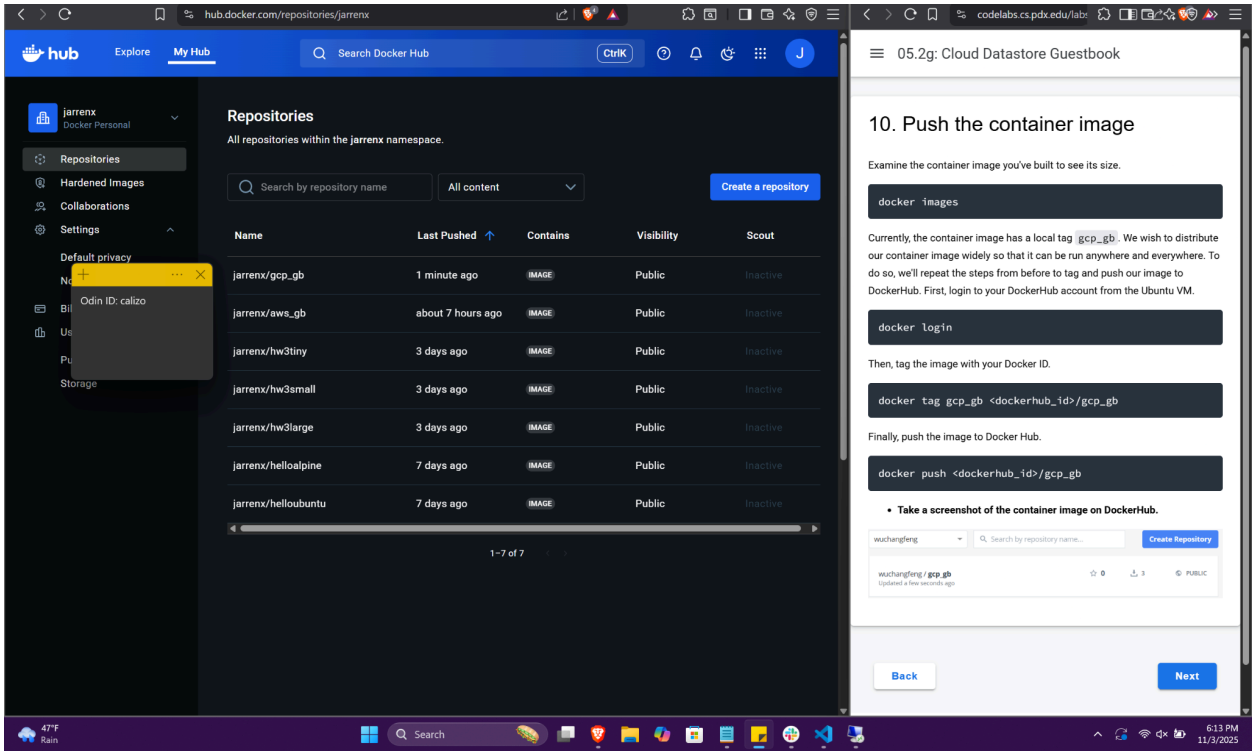
- docker login

Next, I tagged and pushed the image to my public repository:

- docker tag gcp_gb jarrenx/gcp_gb
- docker push jarrenx/gcp_gb

The push completed successfully, and I verified the image’s presence on my Docker Hub account.

1. Take a screenshot of the container image on DockerHub.



The screenshot shows the new repository jarrenx/gcp_gb listed among my public images, timestamped as “1 minute ago,” confirming that the container image was published and is now accessible globally for reuse or redeployment.

<u>Insight</u>

These steps represented the crystallization of deployment maturity - from manual setup to containerized portability. By encapsulating the application and its environment into a single immutable image, the Guestbook evolved into something replicable, auditable, and scalable. Seeing both "Hello Datastore" and "Hello Docker Datastore!" appear in Google Cloud Datastore embodied the harmony between application logic, data persistence, and modern DevOps practice. Each entry symbolized not only successful authentication and persistence but the journey of transforming development work into a platform-agnostic service, ready for the cloud frontier.

Step 11: Version 3: GCP Cloud Shell

To bridge development and deployment, I transitioned the Guestbook application into Google Cloud Shell, a web-based IDE directly integrated with GCP. Its convenience lies

in preconfigured permissions that inherit my account's access scope, removing the need for manual credential setup.

I began by cloning the course repository and navigating into the Datastore directory:

```
git clone https://github.com/wu4f/cs430-src
cd cs430-src/05_gcp_datastore
```

After launching the full-screen terminal window, I opened the Cloud Shell Editor using the pencil icon, allowing simultaneous file editing and terminal execution. Within the gbmodel/model_datastore.py file, I replaced the placeholder YOUR_PROJECT_ID with my actual GCP project identifier obtained using:

- echo $GOOGLE_CLOUD_PROJECT

This ensured the Datastore client was correctly bound to my environment. Saving this change marked the shift from a general template to a fully integrated Cloud Shell version of the Guestbook app.

Step 12: Run the Application

Inside the terminal, I created an isolated environment for dependency management:

```
virtualenv -p python3 env
source env/bin/activate
pip install -r requirements.txt
```

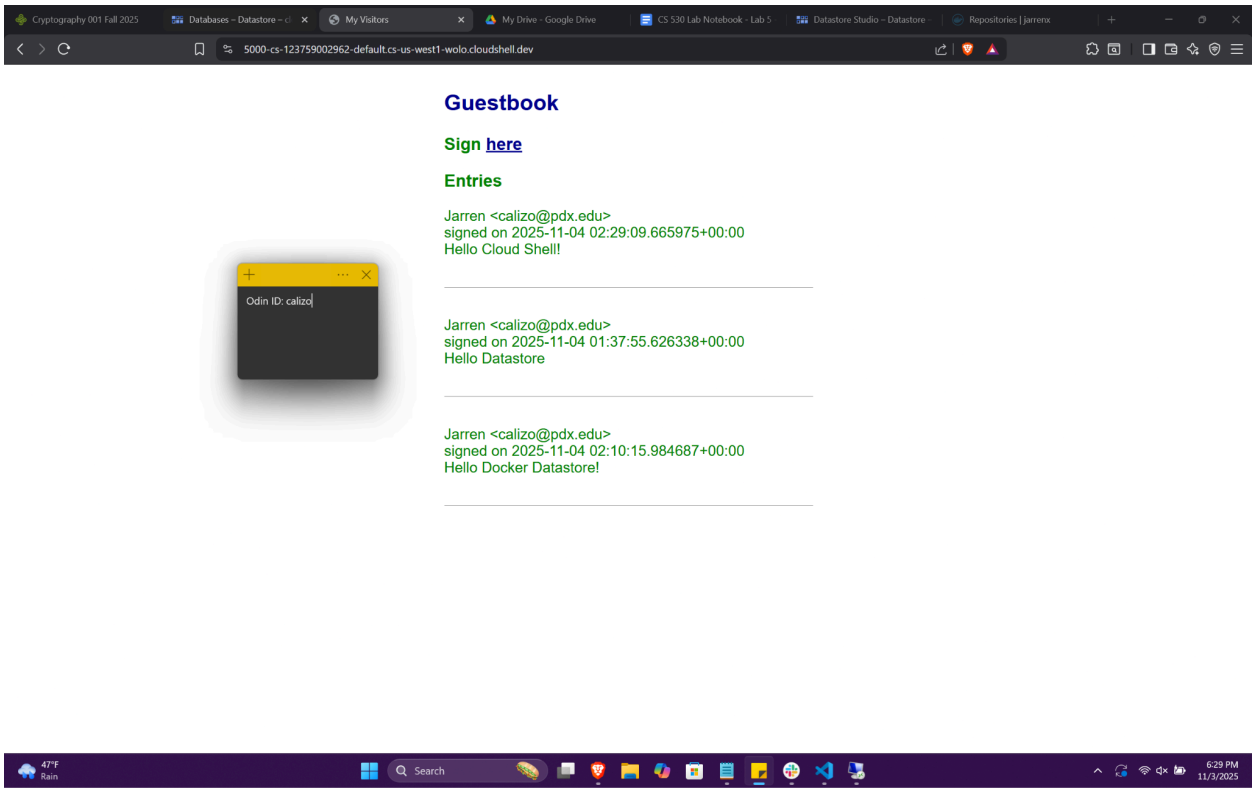
With the setup complete, I launched the Flask application:

- python app.py

The console confirmed the local server running on http://127.0.0.1:5000. Clicking the Web Preview icon and selecting Preview on port 5000 opened a browser tab displaying the familiar Guestbook interface - this time hosted directly on Google Cloud infrastructure.

When prompted to authorize project access, I granted Cloud Shell permission, allowing the app to interact with my Datastore backend seamlessly.

1. Take a screenshot as before that shows your entry and the URL bar.



I signed the guestbook again, using my PSU email and the message "Hello Cloud Shell!". The entry appeared above my previous ones (“Hello Datastore” and “Hello Docker Datastore!”), confirming this version was fully connected to the same backend.

Insight

This stage emphasized the elegance of simplicity in cloud-native workflows. Unlike previous deployments that required manual role assignments and credential mounts, Cloud Shell allows for execution through its built-in permissions and environment setup. Seeing all three entries - "Hello Datastore," "Hello Docker Datastore!," and "Hello Cloud Shell!" - lined up together served as both verification and symbolic progression of the Guestbook’s evolution from local VM to containerized runtime, and finally to a direct cloud-managed environment. Each deployment refined the workflow, reinforcing the principle that cloud infrastructure thrives when configuration becomes invisible and productivity becomes effortless.

Step 13: Version 4: GCP Compute Engine

In the final stage of deployment, the Guestbook application was lifted out of Cloud Shell and placed into a dedicated Compute Engine instance - a true simulation of a

production-grade environment with restricted privileges. This approach demonstrates the principle of least privilege in action: instead of the all-encompassing access that Cloud Shell inherits, this instance was bound to the guestbook service account equipped solely with the Cloud Datastore User role.

I created a new virtual machine instance named guestbook in us-west1-b, using Ubuntu 22.04 LTS as the base image and ensuring HTTP traffic was enabled. The instance was configured to use the guestbook@cloud-calizo-calizo.iam.gserviceaccount.com service account. Once launched, the VM came online successfully with a visible external IP, confirming a clean and scoped setup ready for container deployment.

Step 14: Set up the instance

After SSHing into the instance, the initial step was to install Docker to prepare the environment for the containerized Guestbook image:

```
sudo apt update -y
sudo apt install docker.io -y
```

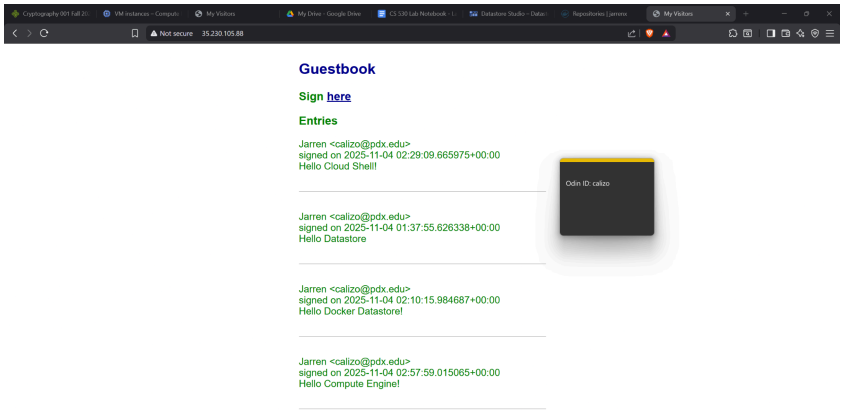
With Docker configured, I pulled and ran my container directly from Docker Hub using privileged mode on port 80:

```
sudo docker run -p 80:80 --env PORT=80 jarrenx/gcp_gb
```

The terminal logs confirmed the image layers were successfully pulled from the Docker registry and that gunicorn began serving the Flask app on port 80. Unlike previous versions, there was no need to mount credentials manually - the attached service account granted seamless authentication to Datastore.

Step 15: Visit the application

1. Take a screenshot as before that shows your entry and the IP address in the URL bar.

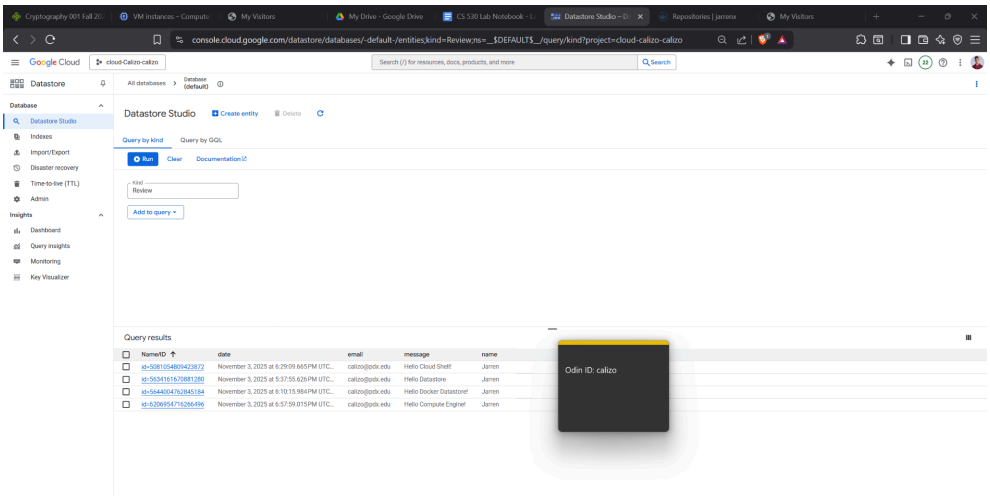


Using the instance’s external IP (35.230.105.88), I accessed the public-facing Guestbook page. The interface rendered perfectly, displaying all prior entries from previous deployment environments. I then added a new entry with the message "Hello Compute Engine!", using my PSU email.

The refreshed page revealed the fourth entry beneath the others - marking the final deployment environment in the full evolution of this Guestbook.

Step 16: View the database

1. Take a screenshot of all of the entries that have been added including their timestamps for your lab notebook.



Returning to the Cloud Datastore console, I verified that the entry was correctly propagated to the backend. Under the Review kind, four entities were visible, each representing an entry from different deployment environments:

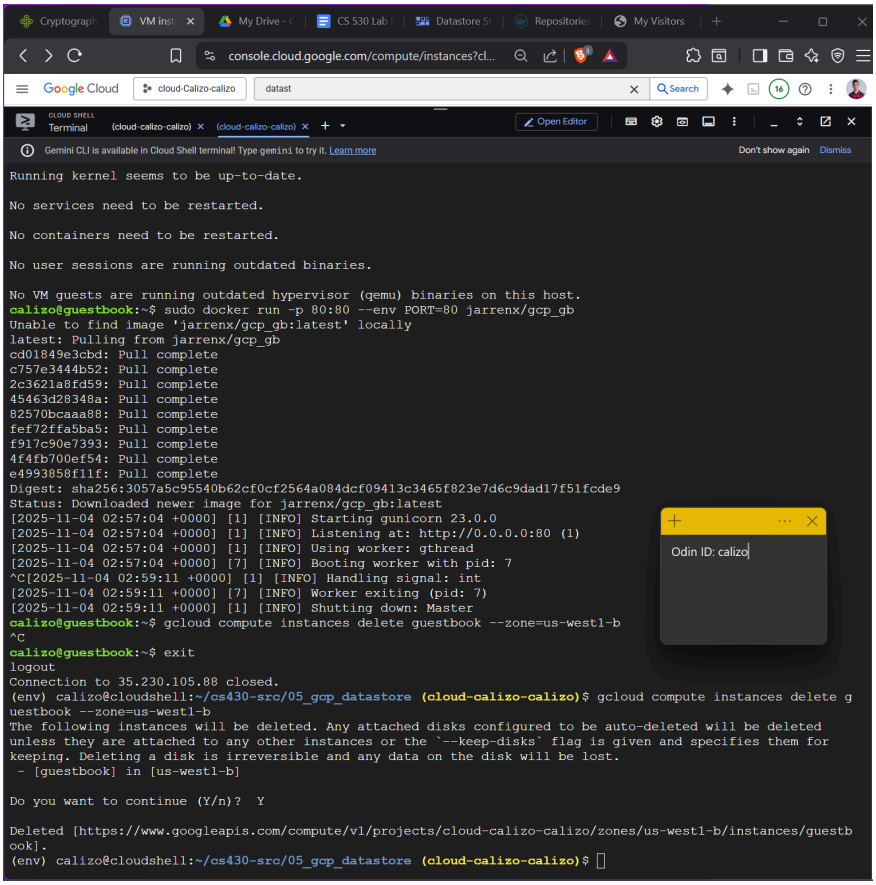
- Hello Datastore (local deployment)
- Hello Docker Datastore! (containerized VM)
- Hello Cloud Shell! (Cloud Shell deployment)
- Hello Compute Engine! (Compute Engine deployment)

Each entry carried a distinct timestamp, affirming proper synchronization between the Compute Engine container and the Datastore service.

Step 17: Clean up

Once validation was complete, I exited the Compute Engine SSH session and executed the cleanup command from Cloud Shell:

- gcloud compute instances delete guestbook --zone=us-west1-b



The terminal output verified successful deletion of the instance, ensuring no residual billing or dangling assets.

Final Insight

This final segment captured the essence of cloud-native deployment - progressively refining control, isolation, and scalability. Each version represented a milestone:

- Local Datastore demonstrated the foundation.
- Docker introduced portability.
- Cloud Shell showcased convenience through managed credentials.
- Compute Engine completed the journey, embodying real-world deployment best practices through controlled IAM roles and infrastructure-level hosting.

Seeing the “Hello Compute Engine!” entry appear alongside the others in a unified Datastore was a rewarding visual - proof of a seamless migration path from development to production.